

Nombre: Nelly Guadalupe Lara Hernández

Fecha: 27 de octubre del 2026

Instrucciones SQL Easy: Documentación de las 31 consultas.

Lección 1 SELECT *

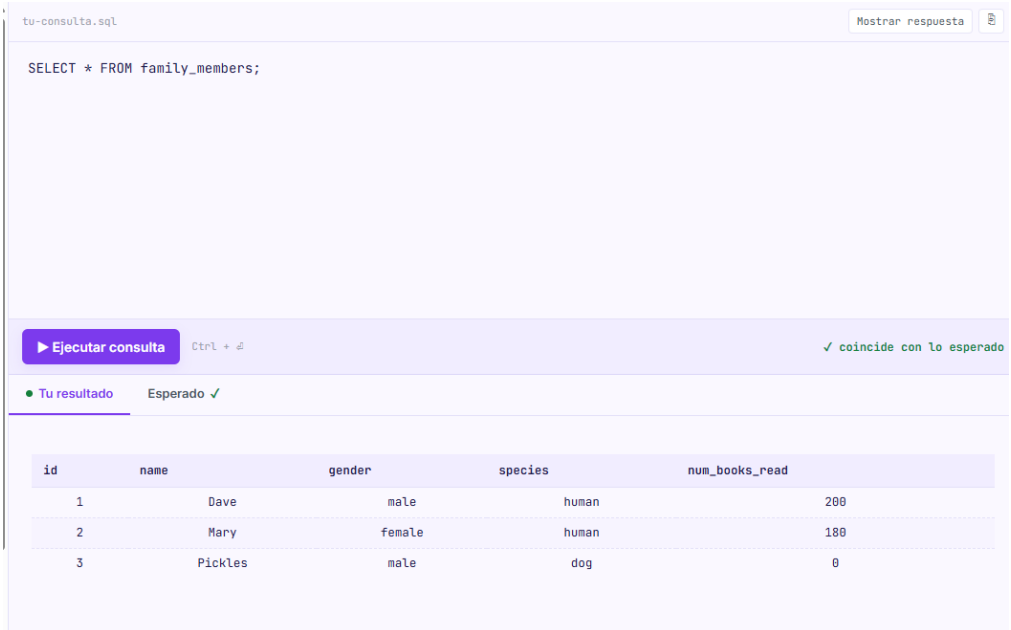
Objetivo del ejercicio:

- ✓ Comprender la estructura elemental de una consulta de lectura en SQL.

Concepto SQL: Consulta básica de datos mediante la instrucción SELECT*FROM

Consulta ejecutada: `SELECT * FROM family_members;`

Resultado:



Interpretación:
La consulta nos devuelve la totalidad de los registros y campos contenidos en la tabla family_members.

Aprendizaje:
En este ejercicio se aprendió sobre la estructura necesaria para la extracción de datos de una base de datos relacional. Con el * pedimos todas las columnas de la tabla sin tener que escribirlas individualmente.

Aplicación en el mundo real:
En el entorno empresarial, este tipo de consulta se emplea habitualmente para realizar inspecciones visuales rápidas, auditar procesos de carga y migración de datos o exportar catálogos maestros completos hacia herramientas de análisis, sirviendo como el punto de partida fundamental antes de aplicar filtros y agregaciones más avanzados.

Lección 2 SELECT columnas especificas

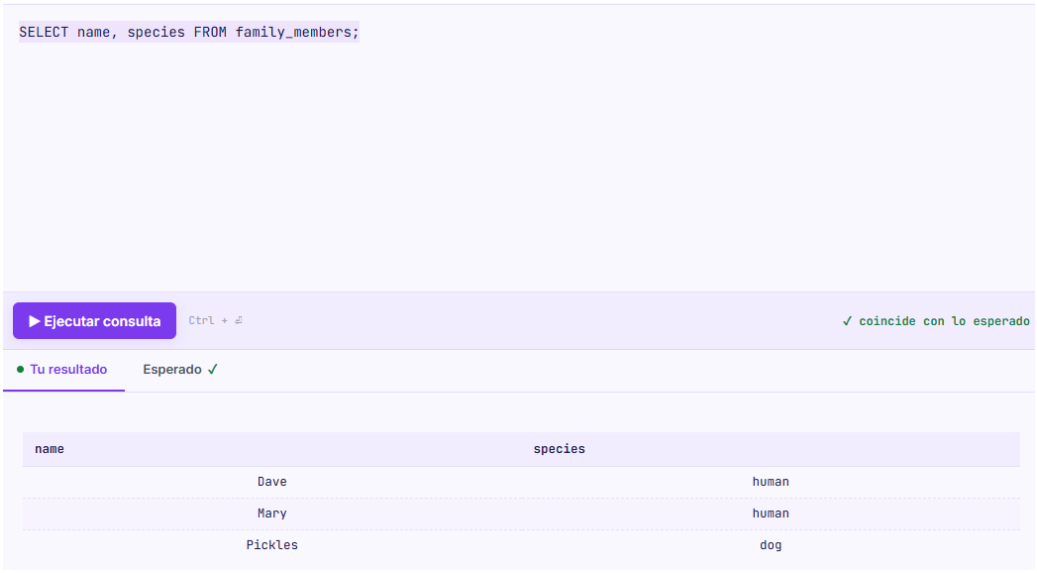
Objetivo del ejercicio:

- ✓ Recuperar de manera selectiva únicamente los campos de nombre y especie (name, species) de todos los registros de la tabla family_members, omitiendo el resto de las columnas no requeridas.

Concepto SQL: Proyección de datos con la cláusula SELECT listando atributos explícitos (separados por comas) en lugar del comodín asterisco (*).

Consulta ejecutada: SELECT name, species FROM family_members;

Resultado:



Interpretación: La consulta devuelve una vista tabular compuesta exactamente por 2 columnas (name y species) y 3 filas correspondientes a Dave (humano), Mary (humana) y Pickles (perro), excluyendo los datos de id, gender y num_books_read.

Aprendizaje: Se comprendió cómo delimitar el volumen horizontal de datos que devuelve el motor de base de datos especificando únicamente las columnas de interés mediante comas, evitando la sobrecarga que produce consultar toda la tabla por defecto con SELECT *.

Aplicación en el mundo real:
En una organización, esta técnica es fundamental para optimizar el rendimiento y la privacidad; por ejemplo, al generar un directorio de empleados se solicitan únicamente `nombre` y `departamento` (evitando transferir por red datos sensibles o innecesarios como salarios, direcciones o números de seguridad social).

Lección 03 WHERE… Equals

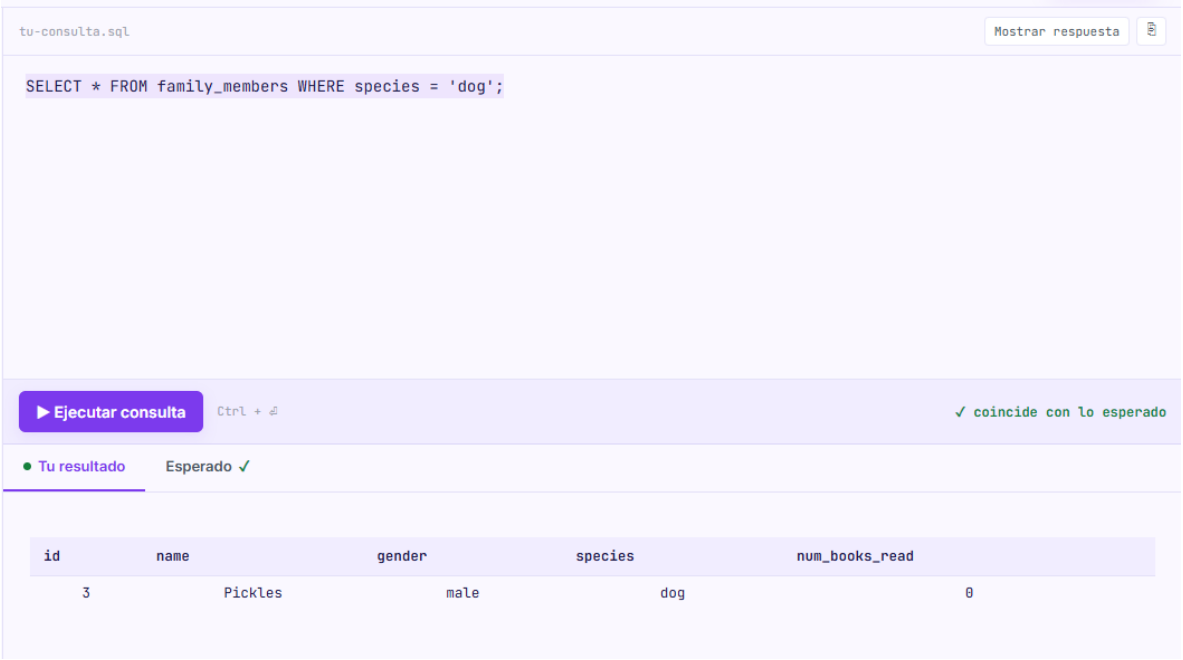
Objetivo del ejercicio:

- ✓ Filtrar los registros de la tabla family_members para obtener todas las filas en las que la especie sea exactamente igual a perro (dog).

Concepto SQL: Filtrado horizontal de registros mediante la cláusula WHERE utilizando el operador de igualdad = con una cadena de texto delimitada por comillas simples ('dog').

Consulta ejecutada: SELECT * FROM family_members WHERE species = 'dog';

Resultado:



Interpretación: La consulta devuelve una única fila con todas sus columnas (id, name, gender, species, num_books_read) correspondiente al miembro con id 3 (Pickles, de género masculino, especie dog y 0 libros leídos), descartando automáticamente a los integrantes humanos.

Aprendizaje: Se aprendió a condicionar la selección de filas a través de criterios específicos con WHERE =, comprendiendo la regla sintáctica de envolver los valores de tipo texto entre comillas simples para distinguirlos de comandos, tablas o columnas del lenguaje SQL.

Aplicación en el mundo real: En las organizaciones se utiliza para segmentar información bajo condiciones concretas; por ejemplo, en una base de datos comercial permite consultar únicamente a los clientes de un país específico (WHERE pais = 'Mexico'), ver las transacciones en un estado particular (WHERE estado = 'Completado') o filtrar empleados asignados a un departamento exacto (WHERE departamento = 'Ventas').

Lección 04 WHERE… Mayor que

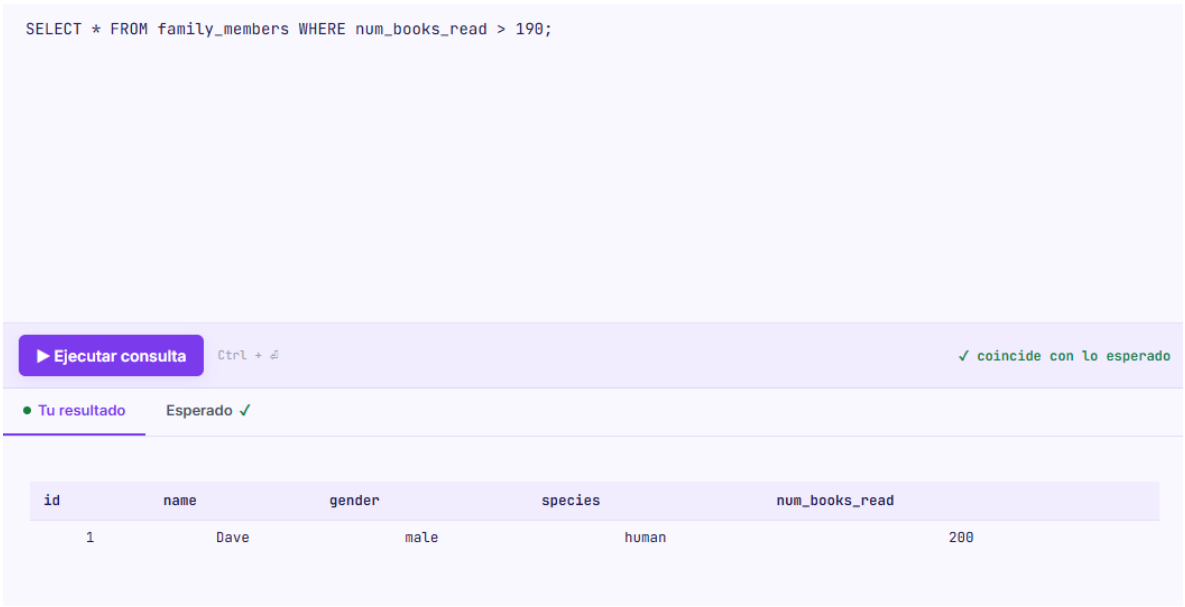
Objetivo del ejercicio:

- ✓ Recuperar todas las filas y columnas de la tabla family_members cuyos miembros tengan un valor en num_books_read estrictamente superior a 190.

Concepto SQL: Filtrado de datos numéricos con la cláusula WHERE utilizando el operador de comparación relacional «mayor que» (>) sin comillas en el valor numérico.

Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read > 190;

Resultado:



Interpretación: La consulta devuelve únicamente la fila correspondiente al registro con id 1 (Dave, masculino, humano, 200 libros leídos). Se excluye a Mary (180 libros leídos, ya que no supera el umbral de 190) y a Pickles (0 libros leídos).

Aprendizaje: Se aprendió a evaluar condiciones cuantitativas sobre campos de tipo entero (int), asimilando la diferencia frente a las cadenas de texto: los números no llevan comillas simples y la comparación es estricta (no incluye valores iguales a 190).
Aplicación en el mundo real: En un entorno empresarial se utiliza para identificar umbrales de métricas o metas de negocio; por ejemplo, encontrar clientes con compras superiores a un monto para programas de fidelidad (WHERE total_gastado > 1000), detectar productos con inventario crítico (WHERE stock < 10) o filtrar pedidos con más de 30 días de retraso.

Lección 05 WHERE… Mayor o igual que

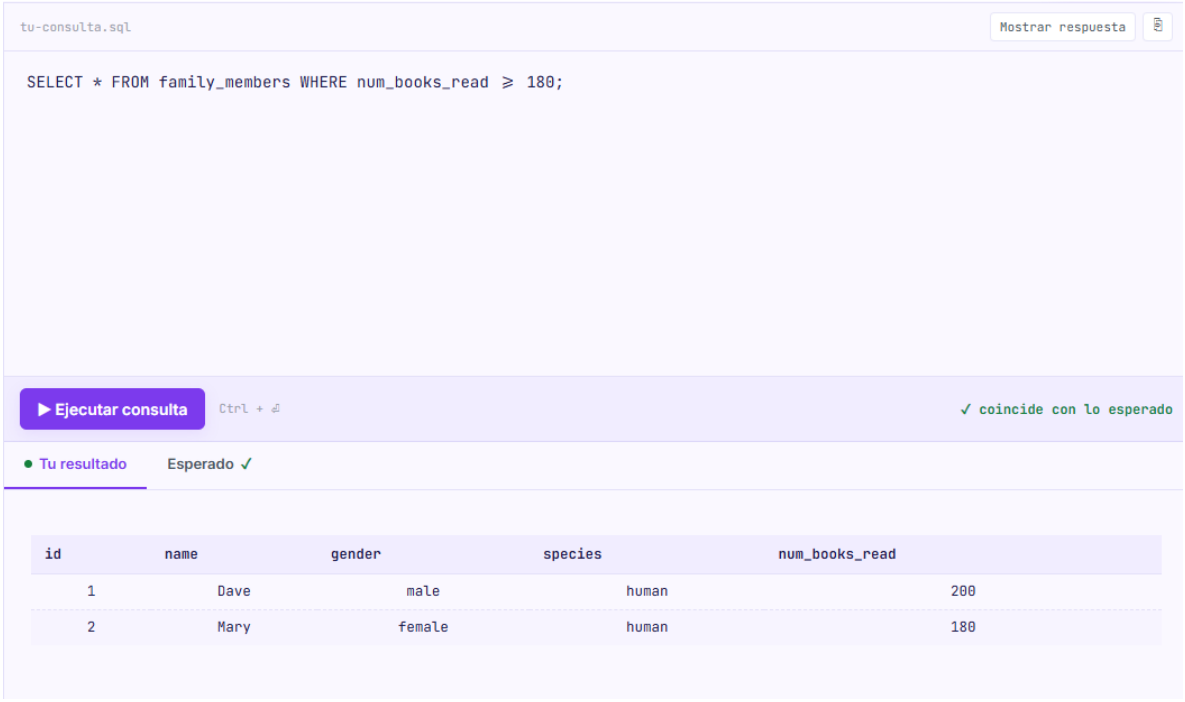
Objetivo del ejercicio:

- ✓ Devolver todas las filas de la tabla `family_members` donde el valor de la columna `num_books_read` sea mayor o igual a 180.

Concepto SQL: Filtrado condicional numérico incluyente mediante la cláusula `WHERE` y el operador de comparación relacional «mayor o igual que» (>= / ≥).

Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read >= 180;

Resultado:



Interpretación:

La consulta devuelve 2 registros con todas sus columnas (id, name, gender, species, num_books_read): Dave con 200 libros leídos y Mary con exactamente 180 libros leídos. Se excluye al miembro canino Pickles por tener 0 libros.

Aprendizaje: Se comprendió la diferencia entre un operador estricto (>) y uno incluyente (>=), entendiendo que al utilizar «mayor o igual» el valor del umbral evaluado (180) sí califica y se incorpora al conjunto final de resultados.

Aplicación en el mundo real: En un entorno empresarial se utiliza para segmentar registros que alcanzan o superan metas o requisitos mínimos; por ejemplo, calificar clientes para un nivel de fidelidad prémium (WHERE compras_acumuladas >= 1000), filtrar postulantes mayores de edad (WHERE edad >= 18) o evaluar empleados que cumplieron con la jornada laboral semanal reglamentaria (WHERE horas_trabajadas >= 40).

Lección 06 AND

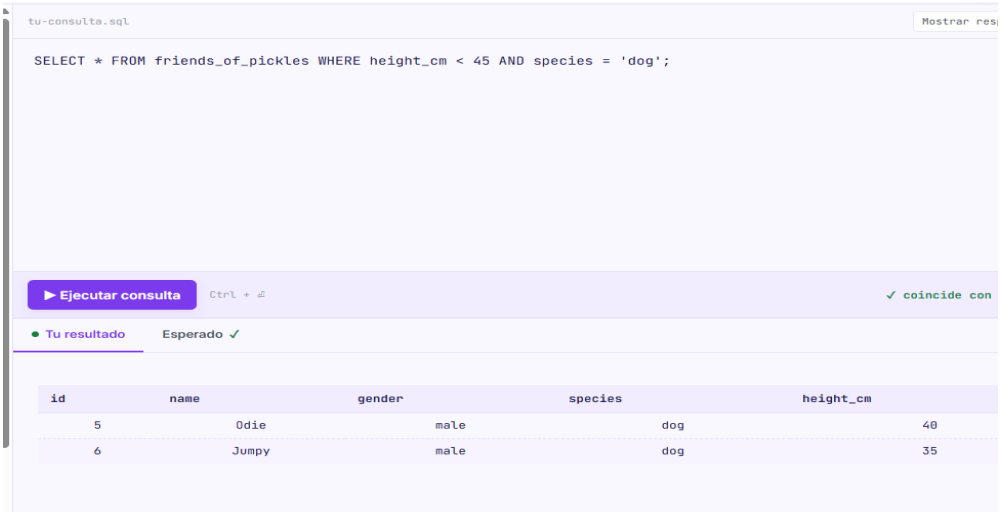
Objetivo del ejercicio:

- ✓ Filtrar la tabla friends_of_pickles para encontrar a todos los amigos que cumplan simultáneamente dos condiciones: ser perros (species = 'dog') y medir menos de 45 cm de altura (height_cm < 45).

Concepto SQL: Filtrado multicriterio en la cláusula WHERE mediante el operador lógico AND, combinando una comparación numérica estricta (<) con una coincidencia de cadena de texto (=).

Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';

Resultado:



Interpretación: La consulta devuelve 2 filas con todas sus columnas (id, name, gender, species, height_cm): Odie (id 5, 40 cm) y Jumpy (id 6, 35 cm). Se descartan los gatos (Fry y Leela), los humanos (Dave y Mary) y el perro Sneakers por medir 55 cm (no cumple la condición de altura menor a 45).

Aprendizaje: Se comprendió el funcionamiento del operador lógico AND, el cual exige que todas las condiciones evaluadas sean verdaderas para que un registro sea incluido en el resultado final, permitiendo refinar búsquedas complejas al cruzar atributos de distinto tipo (texto y número).

Aplicación en el mundo real: En las organizaciones es esencial para realizar segmentaciones precisas cruzando variables; por ejemplo, en logística para identificar paquetes que requieren transporte especial (WHERE peso_kg > 30 AND tipo_envio = 'Fragil'), en recursos humanos para filtrar candidatos (WHERE anos_experiencia >= 3 AND nivel_ingles = 'Avanzado'), o en inventarios para ubicar productos específicos en riesgo (WHERE stock < 10 AND categoria = 'Perecederos').

Lección 07 OR

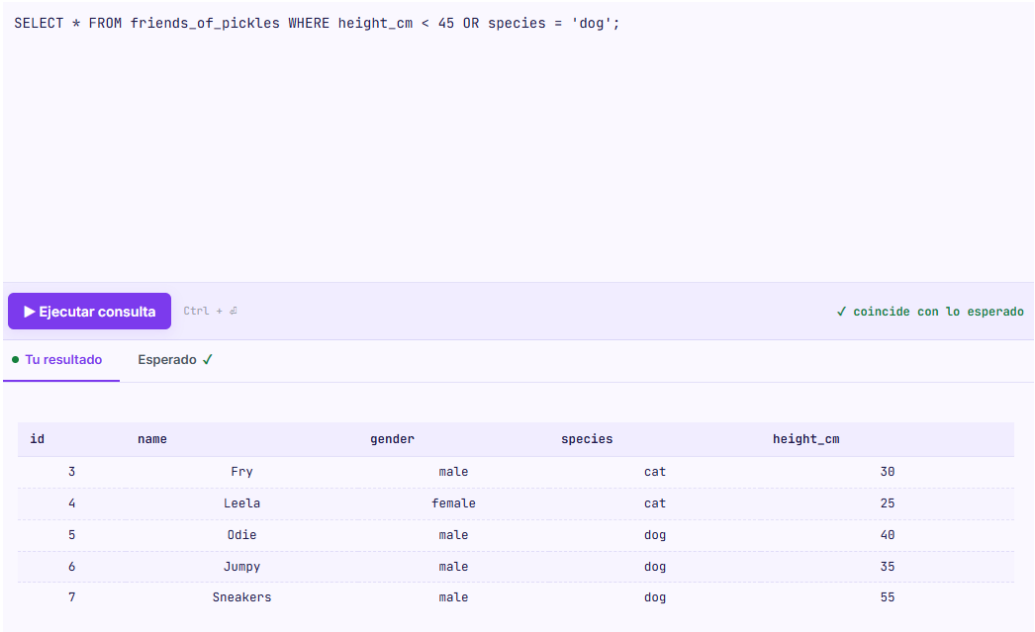
Objetivo del ejercicio:

- ✓ Filtrar la tabla friends_of_pickles para recuperar a todos los amigos que cumplan al menos una de dos condiciones: ser perros (species = 'dog') o medir menos de 45 cm de altura (height_cm < 45).

Concepto SQL: Filtrado condicional disyuntivo en la cláusula WHERE mediante el operador lógico OR, el cual evalúa múltiples criterios y devuelve registros si cualquiera de ellos es verdadero.

Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';

Resultado:



Interpretación: La consulta devuelve 5 registros con todas sus columnas (id, name, gender, species, height_cm): Fry y Leela (califican por medir 30 cm y 25 cm, respectivamente), Odie y Jumpy (califican por ambas condiciones), y Sneakers (califica por ser dog, a pesar de medir 55 cm). Solo se excluyen los humanos Dave y Mary porque no son perros ni miden menos de 45 cm**.**

Aprendizaje: Se comprendió la diferencia fundamental entre los operadores AND y OR: mientras que AND exige cumplir todas las condiciones simultáneamente, OR flexibiliza la búsqueda incluyendo registros que satisfagan al menos uno de los criterios especificados.

Aplicación en el mundo real: En un entorno empresarial se utiliza para ampliar búsquedas o identificar audiencias multicanal; por ejemplo, en marketing para contactar clientes elegibles para una promoción (WHERE ciudad = 'Madrid' OR compras_mensuales > 5), en soporte técnico para listar incidencias prioritarias (WHERE severidad = 'Critica' OR tiempo_espera_horas > 24), o en cobranzas para detectar cuentas en mora (WHERE saldo_pendiente > 0 OR dias_atraso > 30).

Lección 08 IN

Objetivo del ejercicio:

- ✓ Obtener todas las filas de la tabla friends_of_pickles cuyos miembros pertenezcan a especies distintas de gatos o perros (excluir valores 'cat' y 'dog').

Concepto SQL: Filtrado por exclusión de listas con la cláusula WHERE mediante el operador NOT IN, el cual verifica que el valor de una columna no coincida con ninguno de los elementos definidos dentro del conjunto entre paréntesis.

Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');

Resultado:

```
SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');
```

▶ Ejecutar consulta Ctrl + ⏎ ✓ coincide con lo esperado

● Tu resultado Esperado ✓

id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160

Interpretación: La consulta devuelve 2 filas con todas sus columnas (id, name, gender, species, height_cm): Dave (id 1) y Mary (id 2), ambos de especie human. Se excluyen automáticamente los registros correspondientes a gatos (Fry y Leela) y perros (Odie, Jumpy y Sneakers).

Aprendizaje: Se comprendió cómo simplificar condiciones de exclusión múltiple utilizando NOT IN, evitando tener que encadenar varias cláusulas de desigualdad como species != 'cat' AND species != 'dog', lo que hace la consulta más compacta, legible y fácil de mantener.

Aplicación en el mundo real: En entornos corporativos es muy común para depurar o segmentar reportes descartando categorías específicas; por ejemplo, en finanzas para analizar ventas excluyendo pedidos cancelados o reembolsados (WHERE estado NOT IN ('Cancelado', 'Reembolsado')), en logística para listar envíos fuera de regiones restringidas (WHERE pais_destino NOT IN ('Cuba', 'Iran')), o en recursos humanos para filtrar colaboradores activos excluyendo bajas o licencias (WHERE estatus NOT IN ('Baja', 'Inactivo')).

Lección 09 DISTINCT

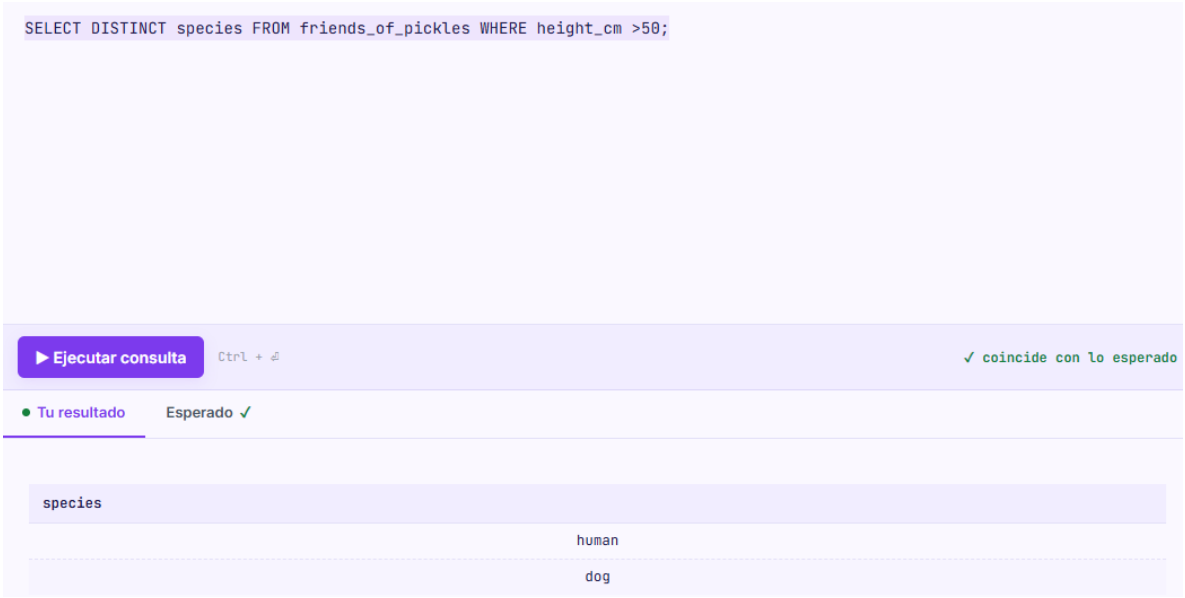
Objetivo del ejercicio:

- ✓ Obtener una lista con los valores únicos o no repetidos de la columna species para todos los animales y miembros de la tabla friends_of_pickles que midan más de 50 cm de altura.

Concepto SQL: Eliminación de duplicados en la proyección mediante la cláusula SELECT DISTINCT, combinada con un filtro numérico condicional estricto en la cláusula WHERE.

Consulta ejecutada: SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm >50;

Resultado:



Interpretación: La consulta evalúa a los registros con height_cm > 50 (Dave con 180 cm, Mary con 160 cm y Sneakers con 55 cm). Aunque Dave y Mary comparten la especie human, la palabra clave

DISTINCT colapsa los duplicados devolviendo únicamente una fila por cada valor único: human y dog.

Aprendizaje: Se comprendió la utilidad de DISTINCT para limpiar la salida de datos redundantes, entendiendo que actúa sobre el conjunto de columnas seleccionadas para mostrar solo combinaciones o valores únicos sin importar cuántas veces existan en la tabla original.

Aplicación en el mundo real: En las organizaciones es fundamental para extraer catálogos de valores existentes y evitar duplicados en reportes; por ejemplo, listar los países únicos donde se tienen clientes activos (SELECT DISTINCT pais FROM clientes), consultar qué categorías de producto tuvieron ventas este mes (SELECT DISTINCT categoria FROM ventas WHERE fecha >= '2026-01-01'), o identificar los puestos laborales existentes en una empresa (SELECT DISTINCT cargo FROM empleados).

Lección 10 ORDER BY

Objetivo del ejercicio:

- ✓ Ordenar todos los registros de la tabla friends_of_pickles según la columna de altura (height_cm) en forma descendente (del valor más alto al más bajo).

Concepto SQL: Ordenamiento de conjuntos de resultados mediante la cláusula ORDER BY combinada con la palabra clave DESC (descendente).

Consulta ejecutada: SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;

Resultado:

```
SELECT * FROM friends_of_pickles ORDER BY  height_cm DESC;
```

▶ Ejecutar consulta Ctrl + ⏎ ✓ coincide con lo esperado

● Tu resultado Esperado ✓

id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160
7	Sneakers	male	dog	55
5	Odie	male	dog	40
6	Jumpy	male	dog	35
3	Fry	male	cat	30
4	Leela	female	cat	25

Interpretación: La consulta devuelve los 7 registros de la tabla con todas sus columnas, organizados jerárquicamente de mayor a menor altura: Dave (180 cm), Mary (160 cm), Sneakers (55 cm), Odie (40 cm), Jumpy (35 cm), Fry (30 cm) y Leela (25 cm).

Aprendizaje: Se comprendió que, por defecto, SQL ordena de forma ascendente (ASC), por lo que es necesario indicar explícitamente DESC al final del campo si se desea priorizar los valores mayores primero.

Aplicación en el mundo real: En las organizaciones es indispensable para crear clasificaciones y rankings; por ejemplo, identificar a los clientes con mayores compras anuales (ORDER BY total_compras DESC), listar las transacciones más recientes (ORDER BY fecha_transaccion DESC) o visualizar a los empleados con mejores métricas de desempeño.

Lección 11 LIMIT # de filas devueltas

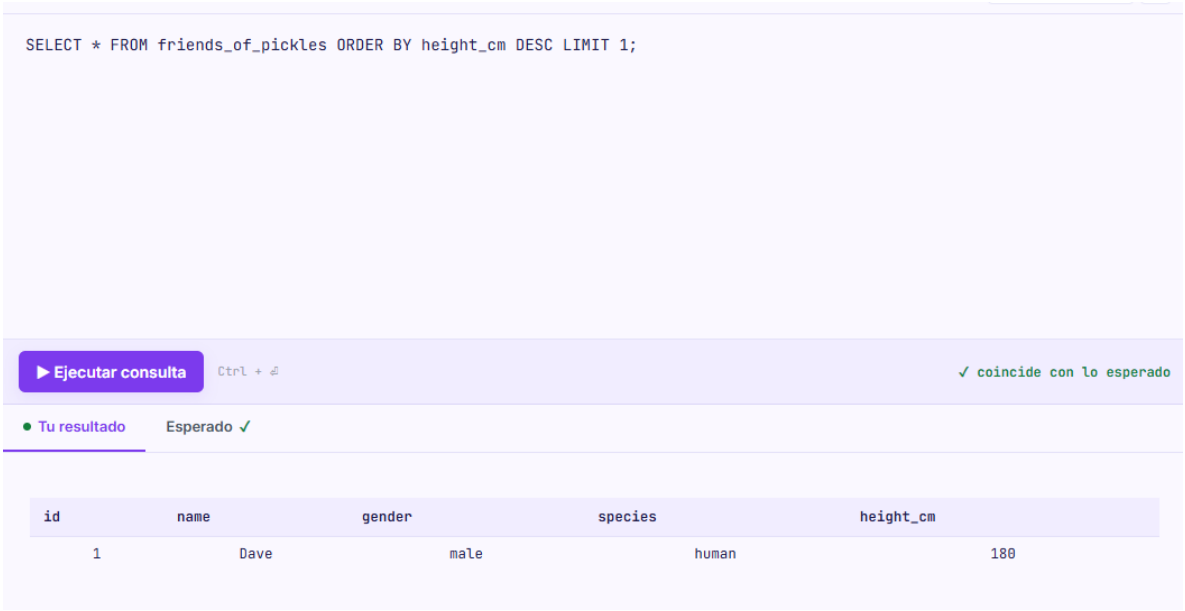
Obejtivo del ejercicio:

- ✓ Devolver una fila única (con todas las columnas) que corresponda al miembro más alto de la tabla friends_of_pickles.

Concepto SQL: Restricción de la cantidad de registros devueltos mediante la cláusula LIMIT, utilizada de forma combinada con el ordenamiento descendente ORDER BY ... DESC.

Consulta ejecutada: SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;

Resultado:



Interpretación: La consulta organiza internamente todos los registros de mayor a menor según height_cm y extrae exclusivamente el primer registro: Dave (id 1, género masculino, especie humana y 180 cm de estatura), descartando las demás filas del resultado final.

Aprendizaje: Se comprendió que la cláusula LIMIT va al final de la consulta y permite recortar el resultado al número exacto de filas solicitado; al combinarla con ORDER BY, es posible obtener valores extremos (como el máximo o el mínimo) de forma directa y eficiente.

Aplicación en el mundo real: En las organizaciones es fundamental para optimizar consultas en tablas con millones de registros y extraer registros destacados; por ejemplo, encontrar el producto más vendido (ORDER BY unidades_vendidas DESC LIMIT 1), visualizar una muestra rápida de datos (LIMIT 10), paginar resultados en aplicaciones web o identificar la transacción financiera de mayor valor del día.

Lección 12 COUNT (*)

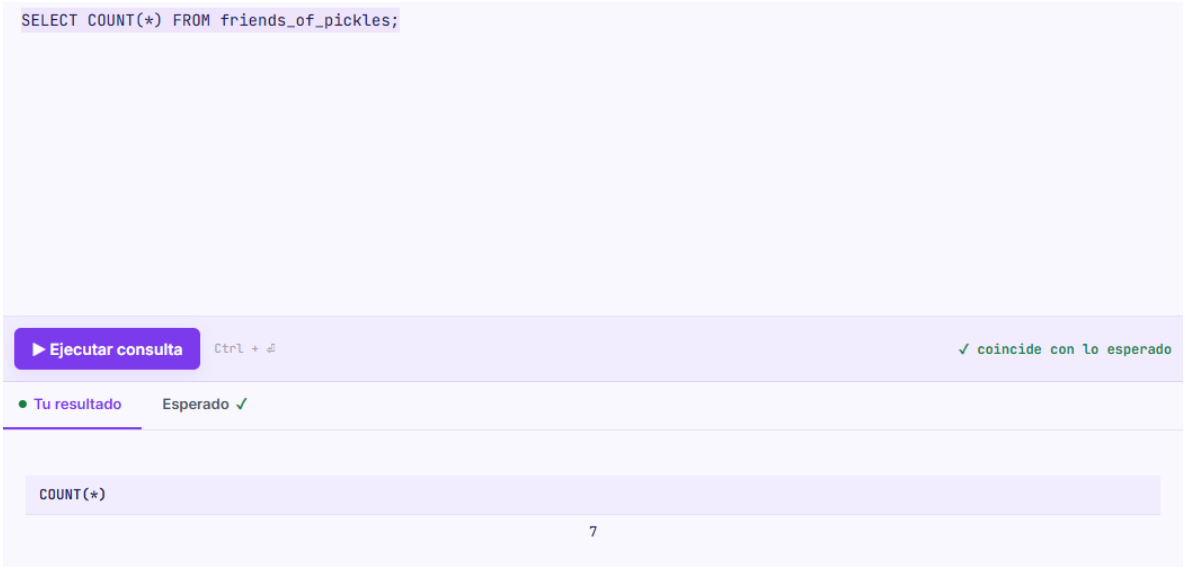
Objetivo del ejercicio:

- ✓ Calcular y devolver el número total de registros (filas) que contiene la tabla friends_of_pickles.

Concepto SQL: Uso de funciones de agregación mediante COUNT(*), la cual contabiliza el total de filas devueltas por una consulta, independientemente de los valores nulos.

Consulta ejecutada: SELECT COUNT(*) FROM friends_of_pickles;

Resultado:



Interpretación: La consulta no devuelve las columnas habituales ni los datos individuales de los amigos, sino una única celda con el valor agregado 7, que representa la cantidad total de registros almacenados en la tabla.

Aprendizaje: Se comprendió que las funciones de agregación como COUNT(*) compactan múltiples filas de datos en un solo resultado escalar cuantitativo, siendo una forma directa de medir el volumen de una tabla sin necesidad de listar o procesar manualmente cada registro.

Aplicación en el mundo real: En las organizaciones es una de las funciones más utilizadas para métricas y control de calidad; por ejemplo, para conocer el total de usuarios registrados en una

plataforma (SELECT COUNT(*) FROM usuarios;), auditar el volumen de transacciones procesadas en el día o verificar rápidamente si la carga de un archivo masivo importó la cantidad esperada de registros.

Lección 13 COUNT (*)…WHERE

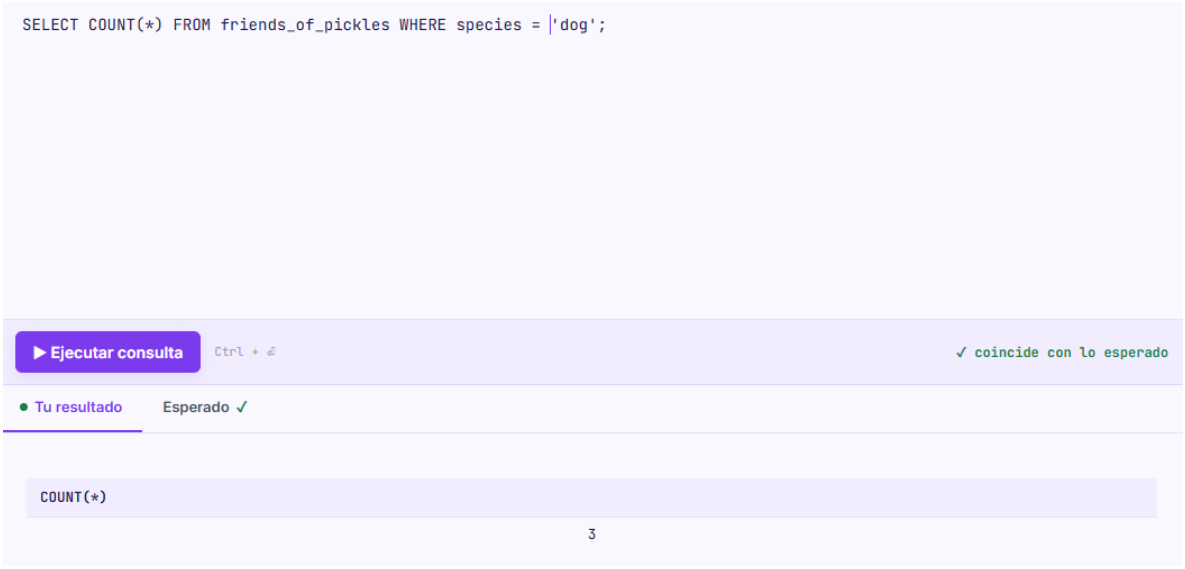
Objetivo del ejercicio:

- ✓ Obtener el número total de filas dentro de la tabla friends_of_pickles donde la especie sea exactamente un perro (species = 'dog').

Concepto SQL: Conteo condicional mediante la combinación de la función de agregación COUNT(*) y la cláusula de filtrado WHERE con igualdad de cadena de texto.

Consulta ejecutada: SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';

Resultado:



Interpretación: El motor de base de datos primero filtra las filas que cumplen la condición species = 'dog' (Odie, Jumpy y Sneakers) y luego aplica la función agregada, devolviendo un valor escalar único de 3 bajo la columna calculada COUNT(*).

Aprendizaje: Se comprendió el flujo de ejecución de SQL al mezclar filtrado y agregación: la cláusula WHERE evalúa y reduce el conjunto de datos de entrada antes de que COUNT(*) compute el total de registros finales.

Aplicación en el mundo real: En las organizaciones es una consulta fundamental para generar indicadores de desempeño (KPIs) y métricas operativas; por ejemplo, calcular el número de clientes dados de baja en el mes (WHERE estatus = 'Inactivo'), contabilizar las ventas realizadas en un canal específico (WHERE canal_venta = 'Online') o saber cuántas órdenes de soporte siguen sin resolver (WHERE estado = 'Pendiente').

Lección 14 SUM

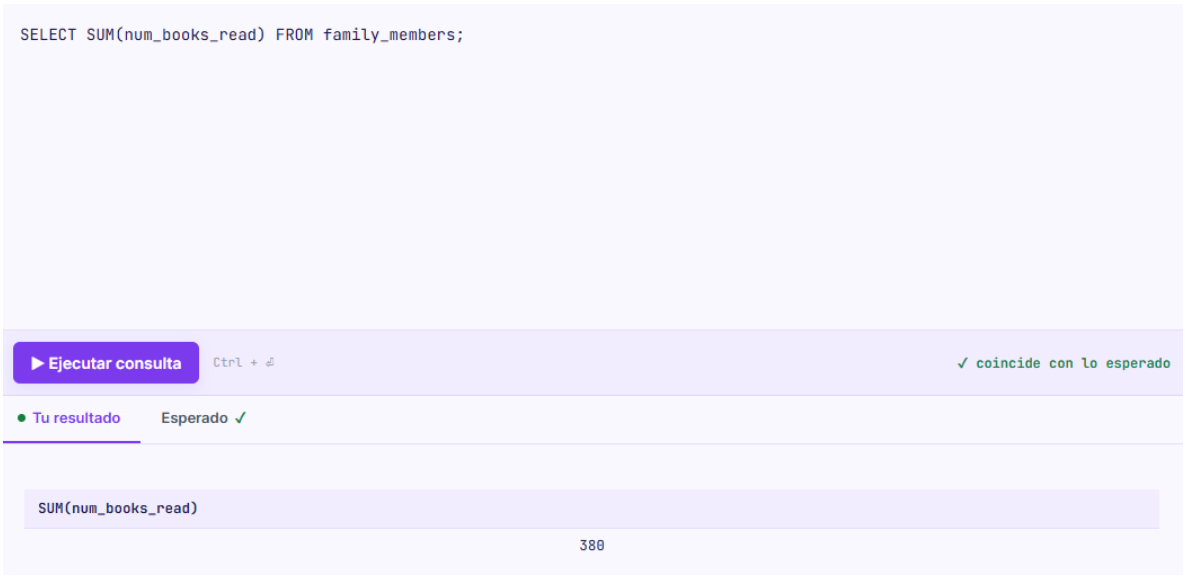
Objetivo del ejercicio:

- ✓ Calcular y devolver la suma total de libros leídos (num_books_read) por todos los integrantes de la tabla family_members.

Concepto SQL: Uso de la función de agregación matemática SUM(), la cual totaliza la suma aritmética de los valores contenidos en una columna de tipo numérico.

Consulta ejecutada: SELECT SUM(num_books_read) FROM family_members;

Resultado:



Interpretación: La consulta procesa todos los valores numéricos de la columna num_books_read (200 de Dave, 180 de Mary y 0 de Pickles) y devuelve un único valor escalar agregado de 380 bajo el campo SUM(num_books_read).

Aprendizaje: Se comprendió la diferencia fundamental entre contar filas (COUNT) y totalizar cantidades (SUM): mientras COUNT devuelve la cantidad de registros evaluados, SUM realiza una operación aritmética sumando el contenido numérico de cada registro en una sola cifra consolidada.

Aplicación en el mundo real: En un entorno empresarial es imprescindible para reportes financieros y operativos clave; por ejemplo, para calcular los ingresos totales por ventas en el año (SELECT SUM(monto) FROM ventas;), conocer el inventario total disponible en bodega (SELECT SUM(unidades_stock) FROM productos;) o sumar las horas extras acumuladas por los trabajadores de un departamento.

Lección 15 AVG

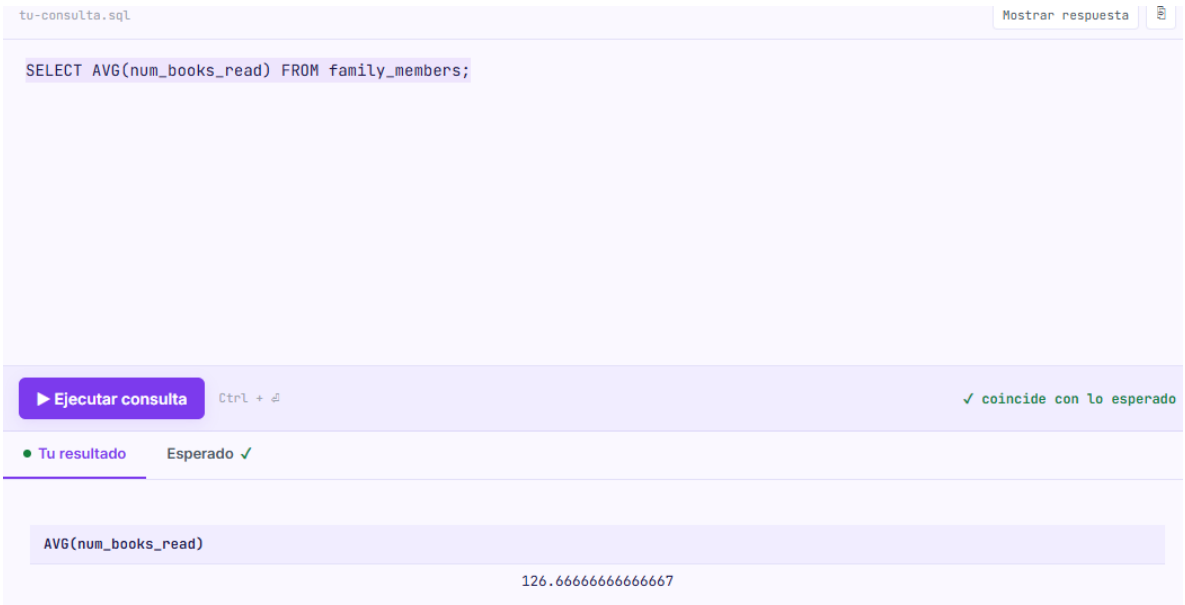
Objetivo del ejercicio:

- ✓ Calcular y devolver el promedio aritmético de libros leídos (num_books_read) por cada miembro registrado en la tabla family_members.

Concepto SQL: Uso de la función de agregación AVG() para computar el promedio o media aritmética sobre una columna de valores numéricos.

Consulta ejecutada: SELECT AVG(num_books_read) FROM family_members;

Resultado:



Interpretación: La consulta toma los valores de num_books_read correspondientes a Dave (200), Mary (180) y Pickles (0), suma el total (380) y lo divide entre los 3 registros existentes, devolviendo un valor escalar con decimales de 126.66666666666667 bajo la columna calculada AVG(num_books_read).

Aprendizaje: Se comprendió que la función AVG() automatiza el cálculo de promedios en un solo paso y que el resultado puede devolver números de punto flotante (con decimales continuos), incluso si los datos originales de la columna son enteros (int).

Aplicación en el mundo real: En un entorno laboral u organizacional es esencial para extraer métricas de desempeño y estándares; por ejemplo, calcular el ticket promedio de compra por cliente (SELECT AVG(monto_total) FROM ventas;), conocer el salario promedio de una empresa o departamento, o evaluar el tiempo medio de resolución de incidencias en soporte técnico.

Lección 16 MAX y MIN

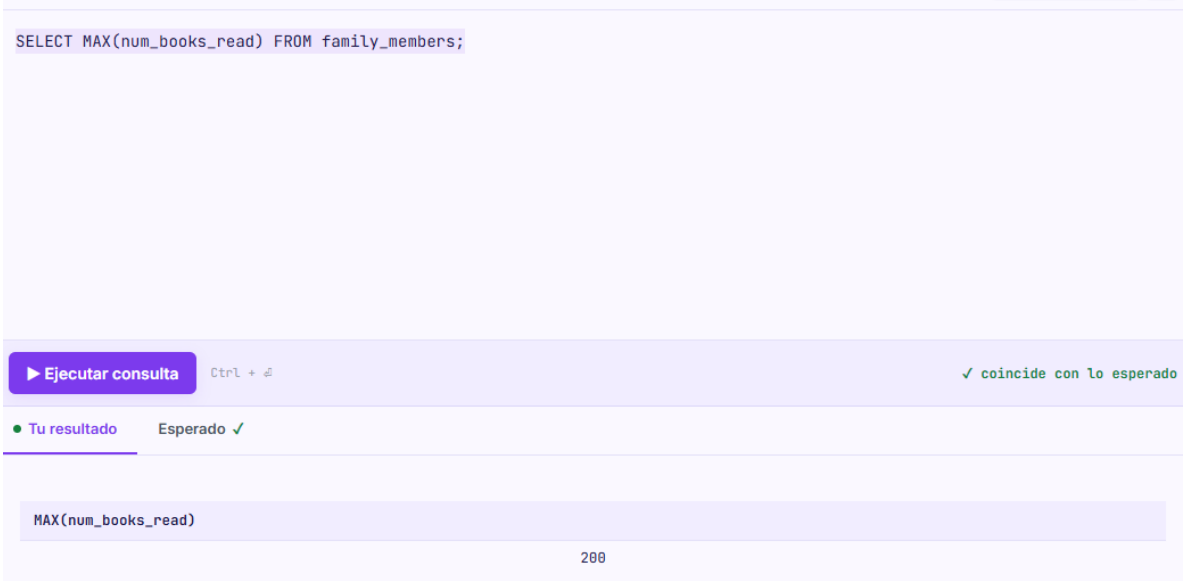
Objetivo del ejercicio:

- ✓ Identificar y devolver el valor más alto registrado en la columna num_books_read entre todos los miembros de la tabla family_members.

Concepto SQL: Uso de funciones de agregación para valores extremos mediante la instrucción MAX(), que localiza el valor máximo contenido en un campo numérico.

Consulta ejecutada: SELECT MAX(num_books_read) FROM family_members;

Resultado:



Interpretación: La consulta analiza los valores de libros leídos de toda la tabla (200, 180 y 0) y devuelve un único resultado escalar con el valor 200 bajo el encabezado calculado MAX(num_books_read).

Aprendizaje: Se comprendió cómo obtener el valor límite superior de un conjunto de datos en un solo paso mediante MAX() sin necesidad de ordenar y limitar filas (ORDER BY ... DESC LIMIT 1), logrando una consulta más concisa y directa para extraer métricas extremas.

Aplicación en el mundo real: En las organizaciones se utiliza para monitorear topes, récords y límites operativos; por ejemplo, encontrar el precio más alto de un catálogo de productos (SELECT MAX(precio) FROM productos;), identificar la mayor venta registrada en el mes o determinar el salario más alto dentro de un departamento.

Lección 17 GROUP BY

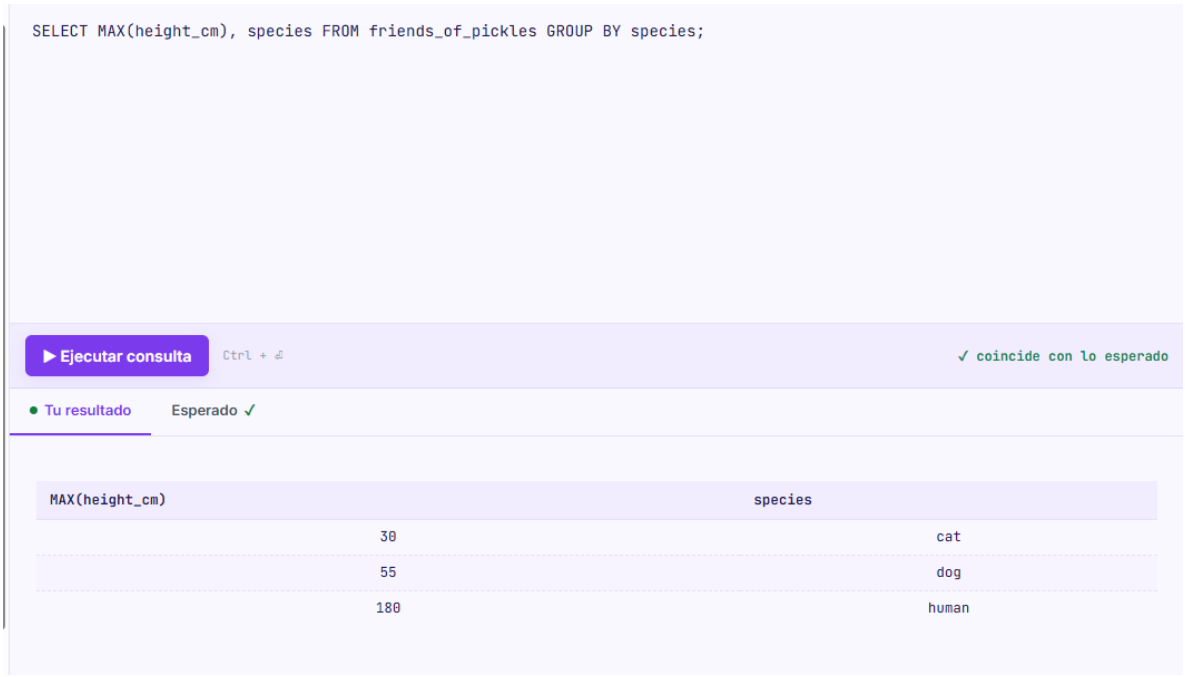
Objetivo del ejercicio:

- ✓ Calcular y devolver la altura máxima (height_cm) registrada para cada especie (species) en la tabla friends_of_pickles, mostrando el nombre de la especie junto a su valor correspondiente.

Concepto SQL: Agrupación de datos mediante la cláusula GROUP BY, combinada con la función de agregación MAX() para calcular métricas resumen por cada subconjunto o categoría.

Consulta ejecutada: SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;

Resultado:



Interpretación: La consulta segmenta las filas de la tabla en tres grupos según la columna species y obtiene el registro más alto de cada categoría: Para cat, la altura máxima es 30 (correspondiente a Fry). Para dog, la altura máxima es 55 (correspondiente a Sneakers). Para human, la altura máxima es 180 (correspondiente a Dave).

Aprendizaje: Se comprendió el rol de GROUP BY para colapsar registros en categorías lógicas y cómo las funciones de agregación (MAX, MIN, AVG, SUM, COUNT) dejan de operar sobre toda la tabla para calcularse de forma independiente dentro de cada grupo definido.

Aplicación en el mundo real: En las organizaciones es una herramienta fundamental para el análisis de datos segmentado; por ejemplo, encontrar el salario más alto pagado por departamento (SELECT departamento, MAX(salario) FROM empleados GROUP BY departamento;), identificar el producto más costoso por categoría en una tienda en línea, o calcular el mayor volumen de ventas por región comercial.

Lección 18 Consultas anidadas

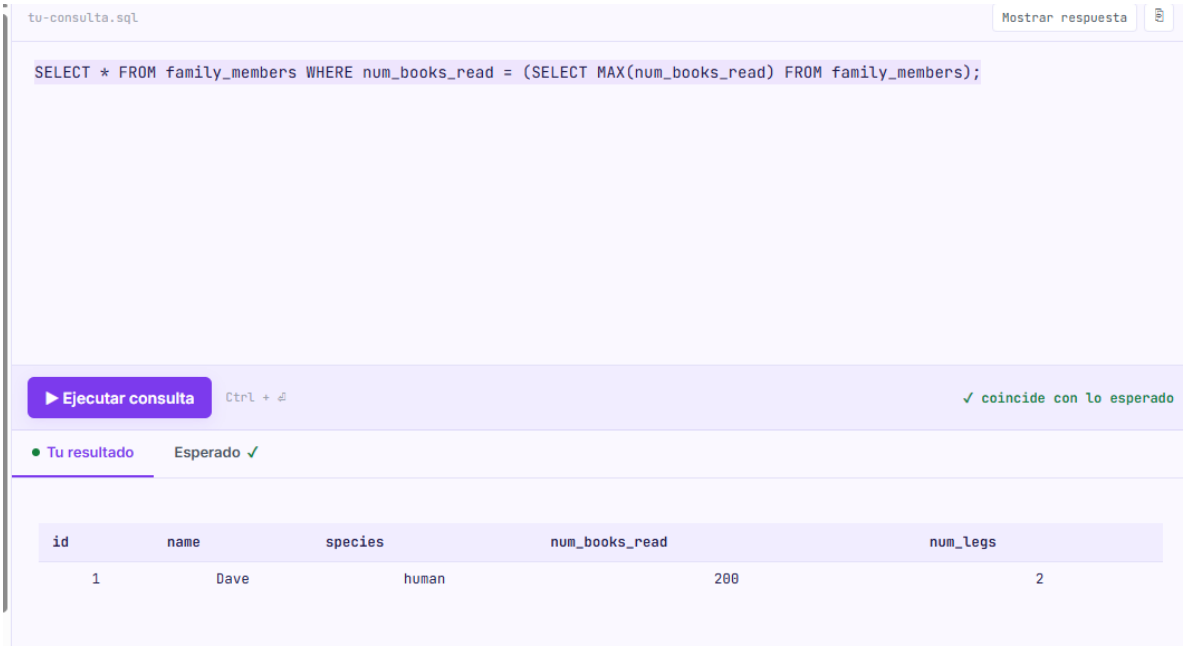
Objetivo del ejercicio:

- ✓ Obtener todas las filas y columnas de los miembros de la tabla family_members que posean la mayor cantidad de libros leídos (num_books_read), calculando dicho valor máximo dinámicamente mediante una subconsulta.

Concepto SQL: Uso de subconsultas (consultas anidadas o subqueries) dentro de la cláusula WHERE, evaluando un valor escalar devuelto por una función de agregación (MAX) contenida entre paréntesis.

Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);

Resultado:



Interpretación: El motor ejecuta primero la consulta interna (SELECT MAX(num_books_read) FROM family_members), la cual devuelve el valor 200. A continuación, la consulta principal busca y devuelve el registro completo correspondiente a Dave (id 1, especie humana, 200 libros leídos y 2 patas), descartando a Mary y Pickles por tener valores inferiores.

Aprendizaje: Se comprendió el orden de evaluación en las consultas anidadas (de adentro hacia afuera) y cómo resolver la limitación de no poder usar directamente una función de agregación en la cláusula WHERE estándar (por ejemplo, escribir WHERE num_books_read = MAX(...) causaría error sintáctico), permitiendo que la subconsulta calcule el valor dinámico primero para que la consulta exterior filtre con él.

Aplicación en el mundo real: En entornos laborales es una técnica fundamental para evitar valores codificados a mano («hardcodeados»); por ejemplo, para encontrar al empleado o empleados con el salario más alto de la empresa (WHERE salario = (SELECT MAX(salario) FROM empleados)), listar los clientes cuya compra más reciente iguala la fecha del último pedido registrado, o extraer a todos los estudiantes cuya calificación esté por encima del promedio general.

Lección 19 NULL

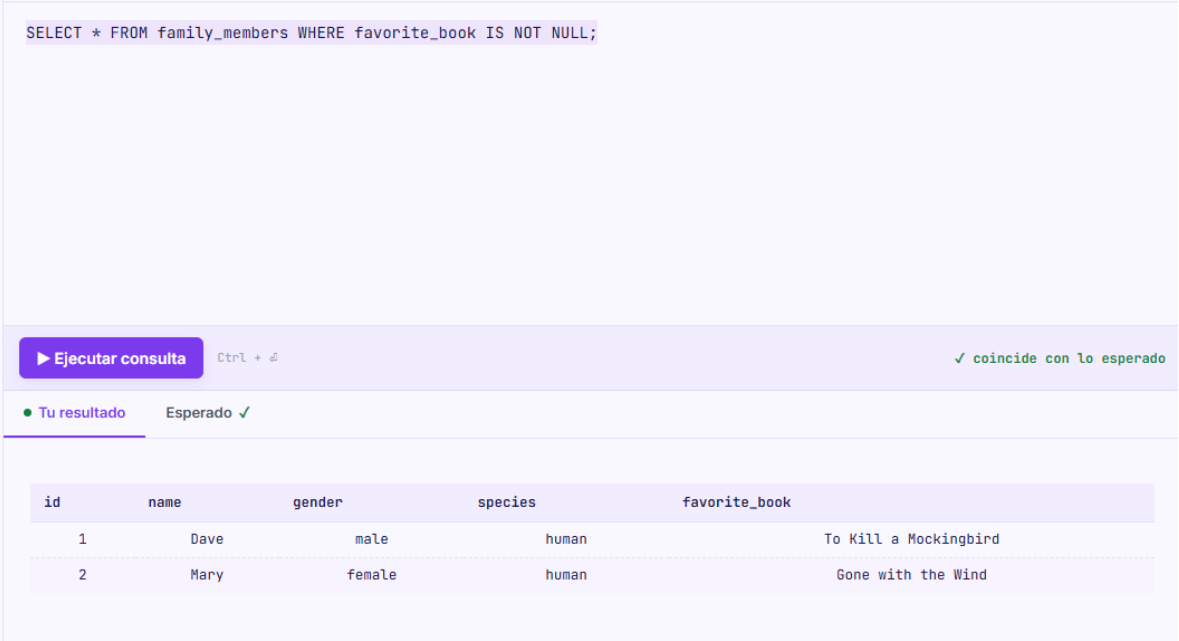
Objetivo del ejercicio:

- ✓ Devolver todas las filas de la tabla family_members donde la columna favorite_book contenga un valor registrado (es decir, que no sea nulo).

Concepto SQL: Manejo y evaluación de valores nulos o ausentes en la cláusula WHERE mediante el operador de comparación especial IS NOT NULL.

Consulta ejecutada: SELECT * FROM family_members WHERE favorite_book IS NOT NULL;

Resultado:



Interpretación: La consulta devuelve 2 filas con todas sus columnas (id, name, gender, species, favorite_book): Dave con el libro "To Kill a Mockingbird" y Mary con "Gone with the Wind". Se descarta a Pickles (id 3) porque el campo favorite_book no contiene ningún dato registrado (NULL).

Aprendizaje: Se comprendió que NULL representa la ausencia total de un valor (no es equivalente a cero ni a un texto vacío) y que en SQL no se pueden utilizar los operadores estándar = o != para evaluarlo, siendo indispensable emplear los operadores especializados IS NULL o IS NOT NULL.

Aplicación en el mundo real: En las organizaciones es crucial para la calidad de datos y auditorías de información; por ejemplo, identificar registros con datos de contacto incompletos (WHERE telefono IS NULL), segmentar usuarios aptos para una campaña de email marketing (WHERE correo_electronico IS NOT NULL) o detectar pedidos que ya fueron entregados al tener una fecha de recepción válida (WHERE fecha_entrega IS NOT NULL).

Lección 20 Fecha

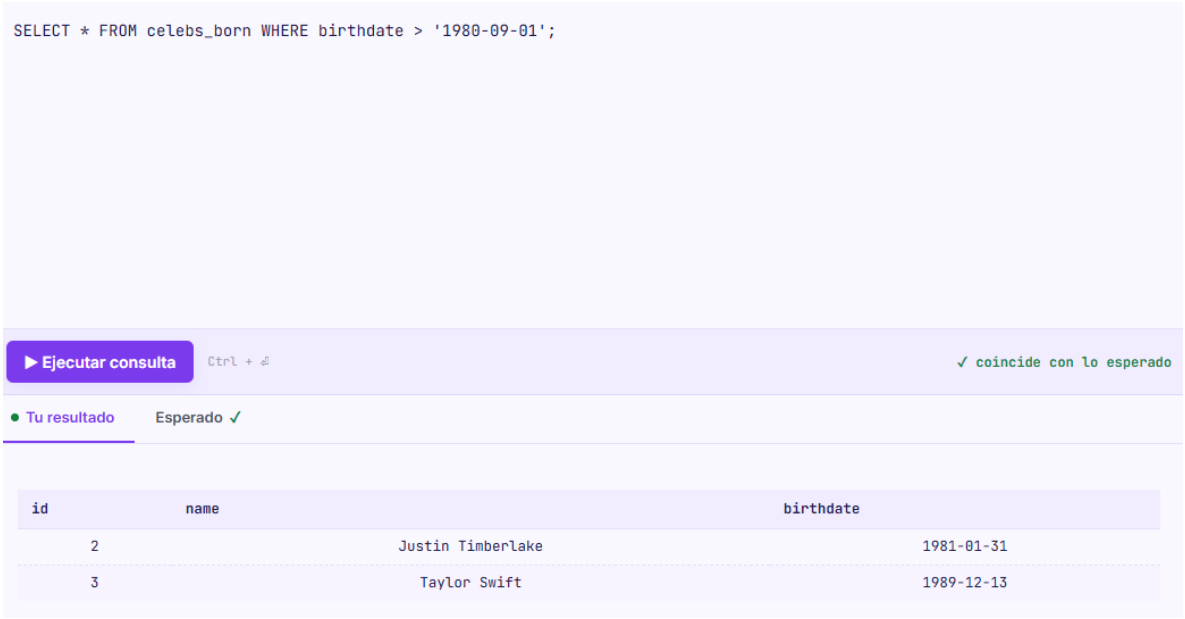
Objetivo del ejercicio:

- ✓ Devolver la lista completa de celebridades de la tabla celebs_born que hayan nacido con posterioridad al 1 de septiembre de 1980.

Concepto SQL: Filtrado temporal y cronológico mediante la cláusula WHERE utilizando el operador relacional «mayor que» (>) aplicado sobre el tipo de dato fecha en formato estándar ISO ('AAAA-MM-DD').

Consulta ejecutada: SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';

Resultado:



Interpretación: La consulta evalúa el campo de fecha de nacimiento (birthdate) y devuelve 2 registros con todas sus columnas (id, name, birthdate): Justin Timberlake (id 2, nacido el 1981-01-31) y Taylor Swift (id 3, nacida el 1989-12-13). Se excluye a Michael Jordan (id 1) por haber nacido antes de la fecha límite (1963-02-17).

Aprendizaje: Se comprendió que en SQL las fechas se escriben entre comillas simples bajo la estructura universal año-mes-día ('YYYY-MM-DD'), y que los operadores relacionales habituales (>, <) funcionan de forma cronológica, donde «mayor que» (>) representa fechas posteriores o más recientes en el tiempo.

Aplicación en el mundo real: En un entorno organizacional es indispensable para generar reportes basados en ventanas temporales; por ejemplo, filtrar transacciones o ventas realizadas a partir de una fecha (WHERE fecha_factura > '2026-01-01'), monitorear contratos por vencer (WHERE fecha_vencimiento > '2026-12-31') o consultar los nuevos empleados contratados en el último período fiscal**.**

Lección 21 Inner Joins

Objetivo del ejercicio:

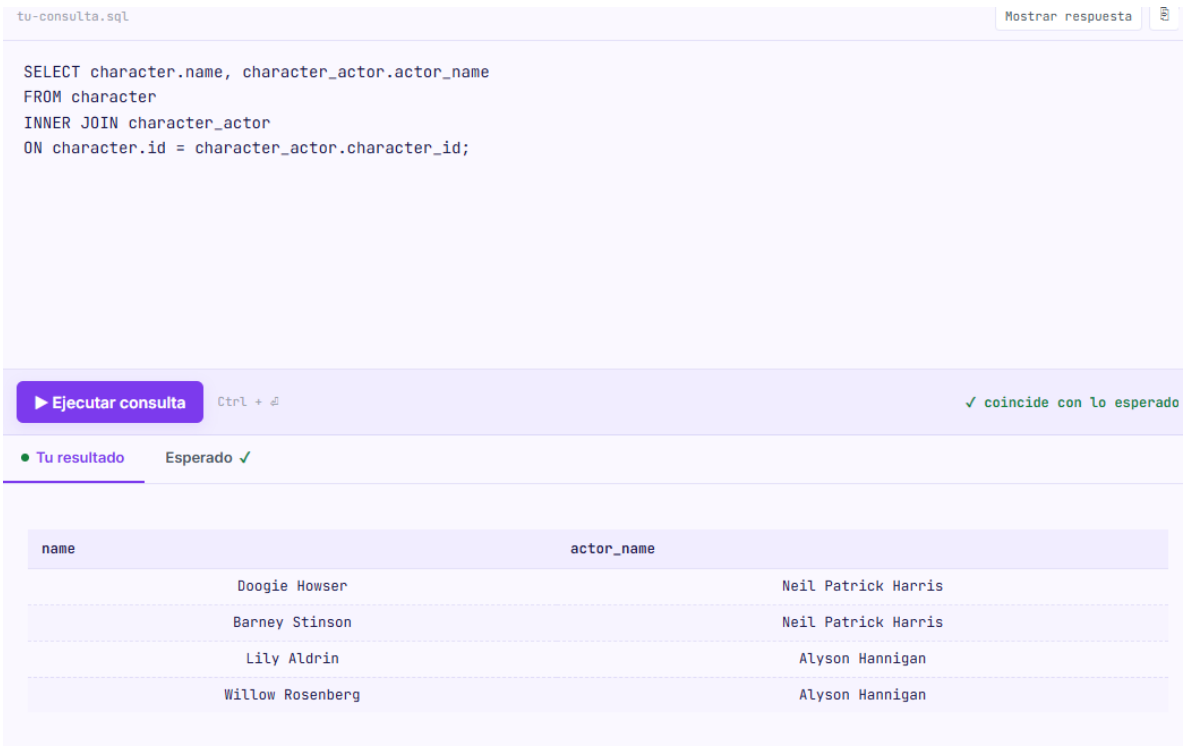
- ✓ Combinar las tablas `character` y `character_actor` para obtener el nombre de cada personaje junto con el nombre del actor que lo interpreta, relacionándolas a través de su identificador común.

Concepto SQL: Unión relacional de tablas mediante la cláusula `INNER JOIN` vinculada a través de la condición de correspondencia `ON` (`clave_primaria = clave_foranea`), empleando la notación de punto (`tabla.columna`) para especificar los campos proyectados.

Consulta ejecutada:

```
SELECT character.name, character_actor.actor_name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id;
```

Resultado:



Interpretación: La consulta vincula el identificador único `character.id` con `character_actor.character_id`, devolviendo una tabla de 2 columnas (`name`, `actor_name`) y 4 filas: Doogie Howser interpretado por Neil Patrick Harris, Barney Stinson por Neil Patrick Harris, Lily Aldrin por Alyson Hannigan, y Willow Rosenberg por Alyson Hannigan.

Aprendizaje: Se comprendió el principio de normalización de bases de datos relacionales para evitar duplicación de información y cómo relacionar tablas separadas utilizando `INNER JOIN ... ON`, así como el uso del prefijo `tabla.campo` para desambiguar atributos y estructurar consultas entre múltiples fuentes.

Aplicación en el mundo real: En las organizaciones es la operación más esencial del modelo relacional, ya que la información se almacena distribuida para mantener la integridad; por ejemplo, cruzar la tabla de pedidos con la de clientes para saber a quién pertenece cada compra (`FROM pedidos INNER JOIN clientes ON pedidos.cliente_id = clientes.id`),

vincular empleados con los nombres de sus departamentos, o asociar facturas con los detalles de sus productos vendidos.

Lección 22 Múltiples Joins

Objetivo del ejercicio:

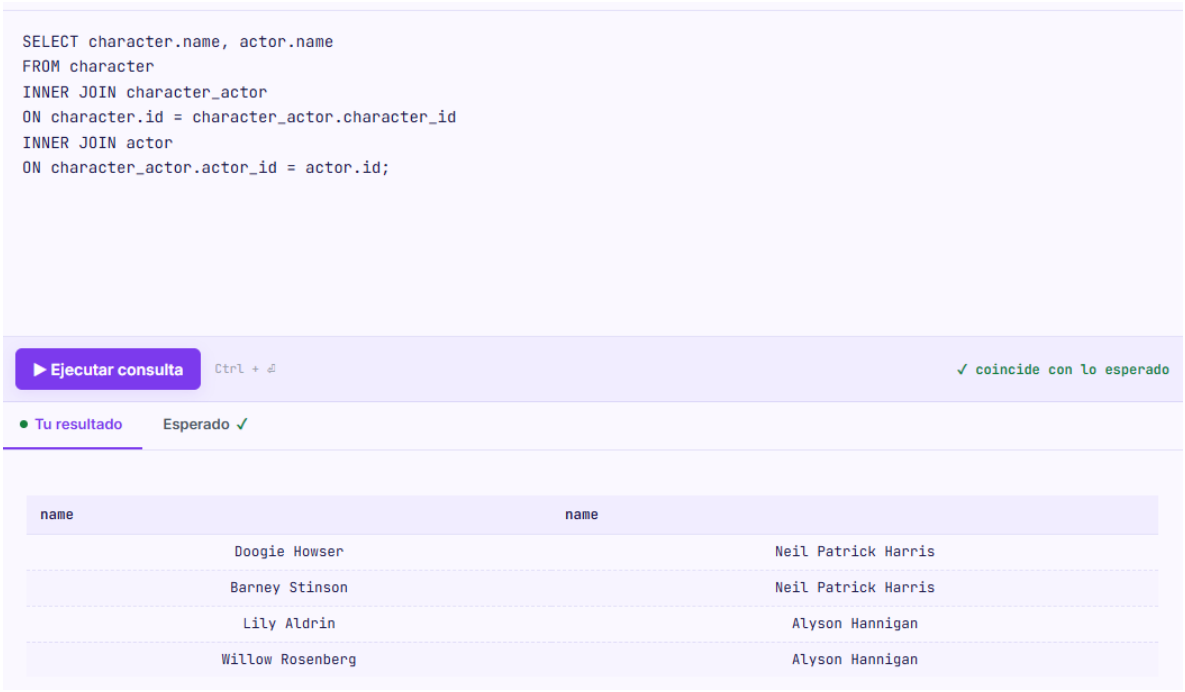
- ✓ Vincular tres tablas distintas (`character`, `character_actor` y `actor`) a través de una tabla intermedia o de unión para obtener el nombre de cada personaje junto al nombre del actor que lo interpreta, sin duplicidad de datos en el esquema.

Concepto SQL: Encadenamiento de múltiples uniones mediante sentencias consecutivas `INNER JOIN ... ON`, resolviendo una relación de muchos a muchos (N:M) a través de una tabla puente o asociativa (`character_actor`).

Consulta ejecutada:

```
SELECT character.name, actor.name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id
INNER JOIN actor
ON character_actor.actor_id = actor.id;
```

Resultado:



name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan

Interpretación: La consulta conecta la entidad `character` con la tabla intermedia `character_actor` mediante `character.id`, y esta a su vez con la entidad `actor` mediante `actor.id`. Devuelve 4 registros donde se visualiza el nombre del personaje y su intérprete correspondiente: Doogie Howser con Neil Patrick Harris, Barney Stinson con Neil Patrick Harris, Lily Aldrin con Alyson Hannigan, y Willow Rosenberg con Alyson Hannigan.

Aprendizaje: Se comprendió la arquitectura de modelado relacional normalizado: en lugar de almacenar texto repetido en tablas intermedias, se utilizan tablas puente con claves foráneas (`character_id`, `actor_id`) y se aplican múltiples `INNER JOIN` sucesivos para recomponer la información completa según sea necesario.

Aplicación en el mundo real: En las organizaciones es el estándar para consultar esquemas totalmente normalizados; por ejemplo, en comercio electrónico para relacionar compras con productos a través de la tabla intermedia de detalles (`pedidos` -> `detalles_pedido` -> `productos`), o en sistemas educativos para vincular estudiantes con los cursos en los que están matriculados (`estudiantes` -> `inscripciones` -> `cursos`).

Lección 23 Joins con WHERE

Objetivo del ejercicio:

- ✓ Obtener una lista con los nombres de personajes y sus respectivos programas de TV relacionando las tablas correspondientes, excluyendo aquellos cuyo personaje sea "Willow Rosenberg" y cuyo programa sea "How I Met Your Mother"

Concepto SQL: Combinación de múltiples cláusulas `INNER JOIN` con filtrado condicional en `WHERE`, utilizando operadores de desigualdad (`!=` o ≠) unidos por el operador lógico `AND`.

Consulta ejecutada:

```
SELECT character.name, tv_show.name

FROM character

INNER JOIN character_tv_show

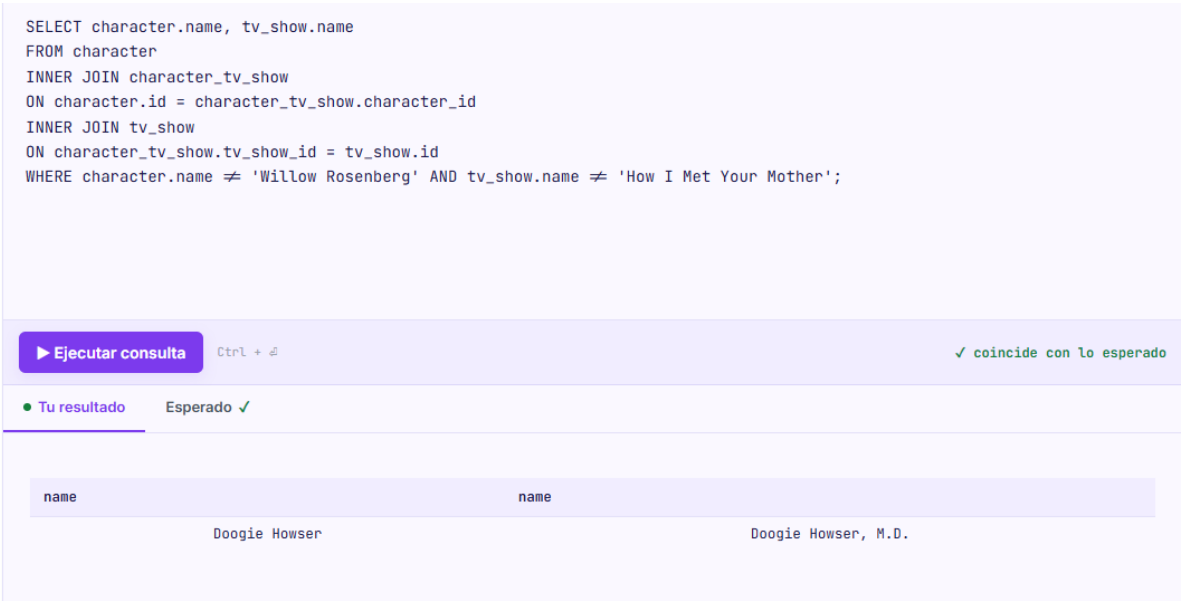
ON character.id = character_tv_show.character_id

INNER JOIN tv_show

ON character_tv_show.tv_show_id = tv_show.id

WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';
```

Resultado:



Interpretación: La consulta une las tablas `character`, `character_tv_show` y `tv_show` para reconstruir la relación entre personaje y programa, y luego aplica el filtro de exclusión. Se descartan todos los registros de "How I Met Your Mother" (como Barney Stinson y Lily Aldrin) y al personaje "Willow Rosenberg", devolviendo únicamente el registro de Doogie Howser perteneciente a la serie "Doogie Howser, M.D.".

Aprendizaje: Se comprendió el orden lógico de procesamiento de consultas relacionales complejas: primero se integran las tablas vinculadas mediante los `INNER JOIN` y sus condiciones `ON`, y posteriormente se evalúan las condiciones restrictivas de la cláusula `WHERE` sobre el conjunto consolidado de columnas de ambas entidades.

Aplicación en el mundo real: En un entorno empresarial permite generar reportes relacionales filtrados descartando casos específicos; por ejemplo, cruzar la tabla de clientes con la de contratos pero excluyendo ciertas regiones y planes de prueba (`WHERE clientes.region != 'Norte' AND planes.tipo != 'Demo'`), o listar pedidos vinculados a proveedores omitiendo transacciones canceladas o de distribuidores internos.

Lección 24 Left joins

Objetivo del ejercicio:

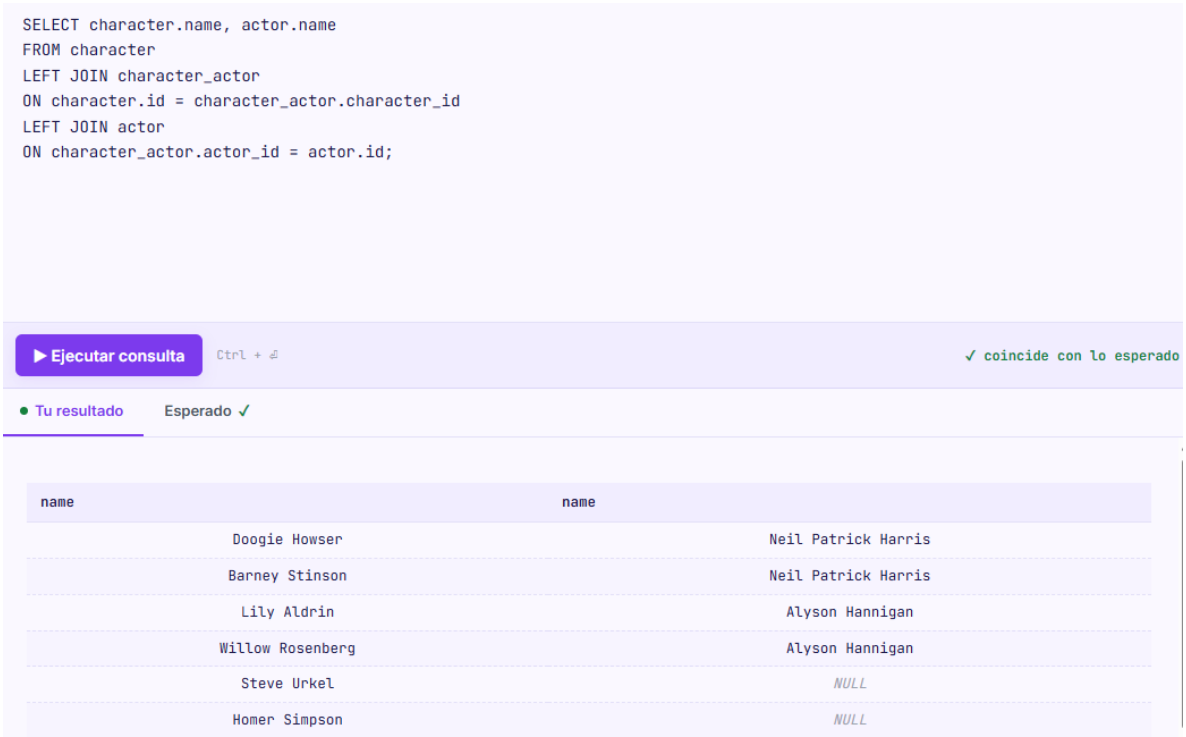
- ✓ Emparejar los nombres de todos los personajes con los nombres de los actores que los interpretan mediante `LEFT JOIN`, preservando a todos los personajes registrados en la tabla principal aunque no tengan un actor asociado en las tablas vinculadas.

Concepto SQL: Uso de uniones externas por la izquierda (`LEFT JOIN ... ON`), donde se devuelven todos los registros de la primera tabla («izquierda») rellenando con valores `NULL` las columnas de las tablas secundarias cuando no existe una coincidencia relacional.

Consulta ejecutada:

```
SELECT character.name, actor.name
FROM character
LEFT JOIN character_actor
ON character.id = character_actor.character_id
LEFT JOIN actor
ON character_actor.actor_id = actor.id;
```

Resultado:



Interpretación: La consulta muestra a los personajes que sí tienen actor registrado (Doogie Howser, Barney Stinson, Lily Aldrin y Willow Rosenberg) junto con su actor correspondiente, e incluye además a "Steve Urkel" y "Homer Simpson", mostrando para ellos el valor `NULL` en el campo del actor al no tener coincidencia en `character_actor`.

Aprendizaje: Se comprendió la diferencia crítica entre `INNER JOIN` y `LEFT JOIN`: mientras que el primero descarta cualquier registro sin correspondencia exacta, el segundo garantiza que ningún dato de la tabla base se pierda, evidenciando ausencias mediante campos `NULL`.

Aplicación en el mundo real: En las organizaciones es esencial para auditorías y detección de registros huérfanos o inactivos; por ejemplo, listar la totalidad de los clientes y sus últimas compras para detectar quiénes jamás han comprado (`FROM clientes LEFT JOIN pedidos ... WHERE pedidos.id IS NULL`), verificar qué productos del catálogo nunca han tenido

movimientos de inventario, o listar todos los empleados indicando quiénes aún no tienen un departamento o jefe asignado.

Lección 25 Alias de tabla

Objetivo del ejercicio:

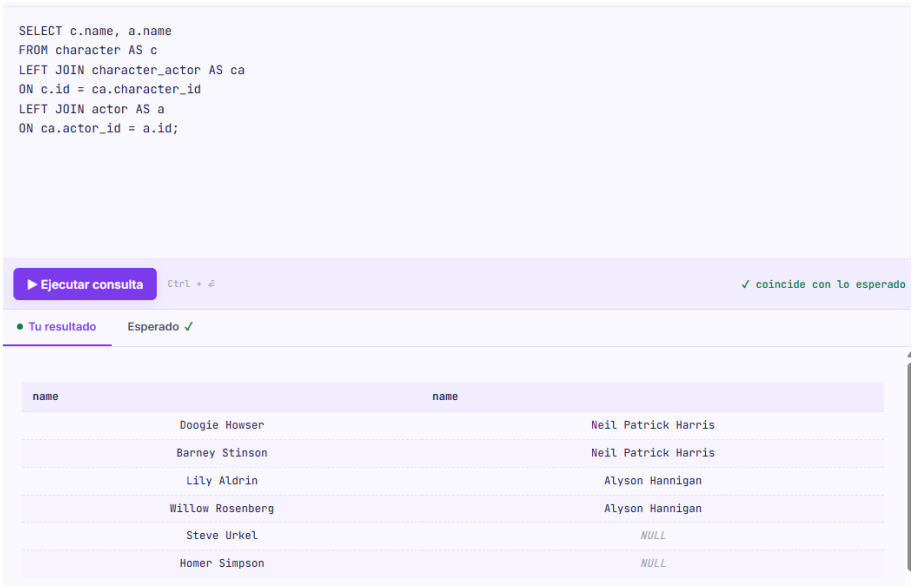
- ✓ Emparejar los nombres de los personajes con sus respectivos actores utilizando `LEFT JOIN` y simplificar la sintaxis del código asignando los alias `c`, `ca` y `a` a las tablas correspondientes.

Concepto SQL: Uso de alias de tablas mediante la palabra clave `AS` para renombrar de forma temporal y abreviada las entidades dentro de la consulta, aplicados en sentencias `LEFT JOIN ... ON` y en la proyección de columnas (`c.name`, `a.name`).

Consulta ejecutada:

```
SELECT c.name, a.name
FROM character AS c
LEFT JOIN character_actor AS ca
ON c.id = ca.character_id
LEFT JOIN actor AS a
ON ca.actor_id = a.id;
```

Resultado:



name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	NULL
Homer Simpson	NULL

Interpretación: La consulta produce el mismo resultado que en la lección previa pero con una sintaxis mucho más compacta. Se listan los 6 personajes de la tabla `character`: cuatro con su actor correspondiente (Doogie Howser y Barney Stinson interpretados por Neil Patrick Harris; Lily

Aldrin y Willow Rosenberg por Alyson Hannigan) y dos personajes (Steve Urkel y Homer Simpson) con valor `NULL` al no contar con un actor asociado en la tabla de relaciones.

Aprendizaje: Se comprendió que los alias de tabla creados con `AS` actúan como identificadores temporales durante la ejecución de la consulta, lo que reduce la redundancia al escribir nombres largos de tablas, mejora la legibilidad del código y facilita la referencia a campos ambiguos mediante prefijos cortos.

Aplicación en el mundo real: En entornos laborales es una práctica estándar para mantener consultas limpias y profesionales al trabajar con esquemas relacionales complejos; por ejemplo, unir tablas con nombres extensos (`FROM transacciones_facturacion_electronica AS tfe INNER JOIN clientes_empresariales_cuentas AS cec ON tfe.cliente_id = cec.id`), o al realizar autouniones (*self-joins*) sobre la misma tabla para jerarquías organizacionales (`FROM empleados AS e LEFT JOIN empleados AS jefe ON e.jefe_id = jefe.id`).

Lección 26 Alias de columna

Objetivo del ejercicio:

- ✓ Emparejar los nombres de los personajes con los nombres de los actores mediante `LEFT JOIN`, asignando alias a las columnas proyectadas para que aparezcan identificadas como `character` y `actor` en lugar de compartir el encabezado genérico `name`.

Concepto SQL: Renombramiento explícito de atributos proyectados mediante la cláusula `AS` a nivel de columna (`SELECT columna AS nuevo_nombre`), evitando colisiones y desambiguando encabezados en el conjunto de resultados.

Consulta ejecutada:

```
SELECT character.name AS character, actor.name AS actor

FROM character

LEFT JOIN character_actor

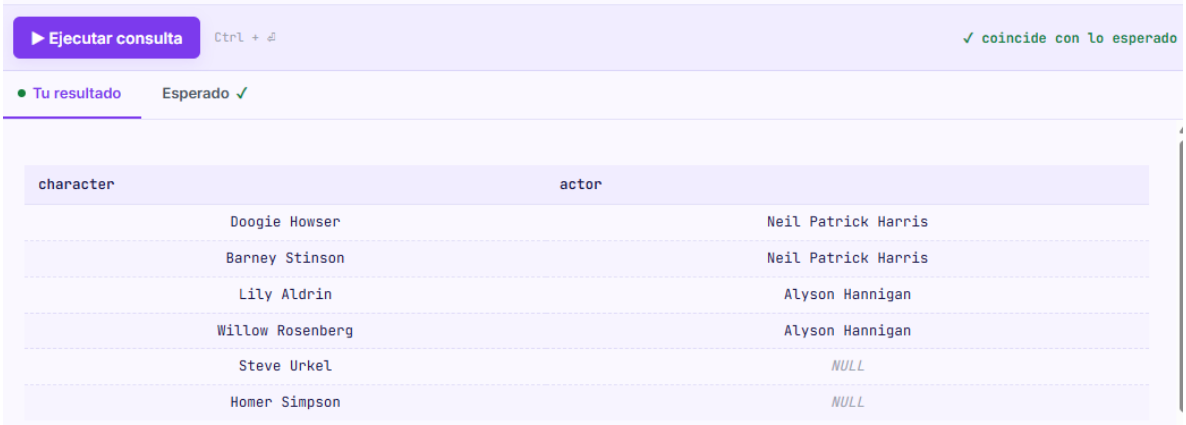
ON character.id = character_actor.character_id

LEFT JOIN actor

ON character_actor.actor_id = actor.id;
```

Resultado:

```
SELECT character.name AS character, actor.name AS actor
FROM character
LEFT JOIN character_actor
ON character.id = character_actor.character_id
LEFT JOIN actor
ON character_actor.actor_id = actor.id;
```



character	actor
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	*NULL*
Homer Simpson	*NULL*

Interpretación: Renombramiento explícito de atributos proyectados mediante la cláusula `AS` a nivel de columna (`SELECT columna AS nuevo_nombre`), evitando colisiones y desambiguando encabezados en el conjunto de resultados.

Aprendizaje: Se comprendió que, al cruzar múltiples tablas que comparten nombres de campos idénticos (como `name`), los alias de columna permiten personalizar la presentación final del informe, logrando claridad semántica sin modificar la estructura original de las tablas en la base de datos.

Aplicación en el mundo real: En las organizaciones es crucial para la generación de reportes limpios y consumo de APIs/dashboards; por ejemplo, al cruzar clientes y ciudades (`SELECT clientes.nombre AS cliente, ciudades.nombre AS ciudad_origen`), al presentar cálculos matemáticos (`SELECT SUM(monto) AS ingresos_totales`), o al preparar datos para herramientas de BI donde las claves deben tener nombres descriptivos.

Lección 27 Self Joins

Objetivo del ejercicio:

- ✓ Realizar una autounión sobre la tabla `employees` para devolver el nombre de cada empleado junto con el nombre de su respectivo jefe, empleando alias para nombrarlos en las columnas como `employee_name` y `boss_name`.

Concepto SQL: Autounión (*self-join*) mediante `INNER JOIN`, donde una tabla se vincula consigo misma a través de una relación recursiva o jerárquica (`clave_foranea = clave_primaria_misma_tabla`), haciendo obligatorio el uso de alias de tabla para diferenciar los dos roles en la consulta.

Consulta ejecutada:

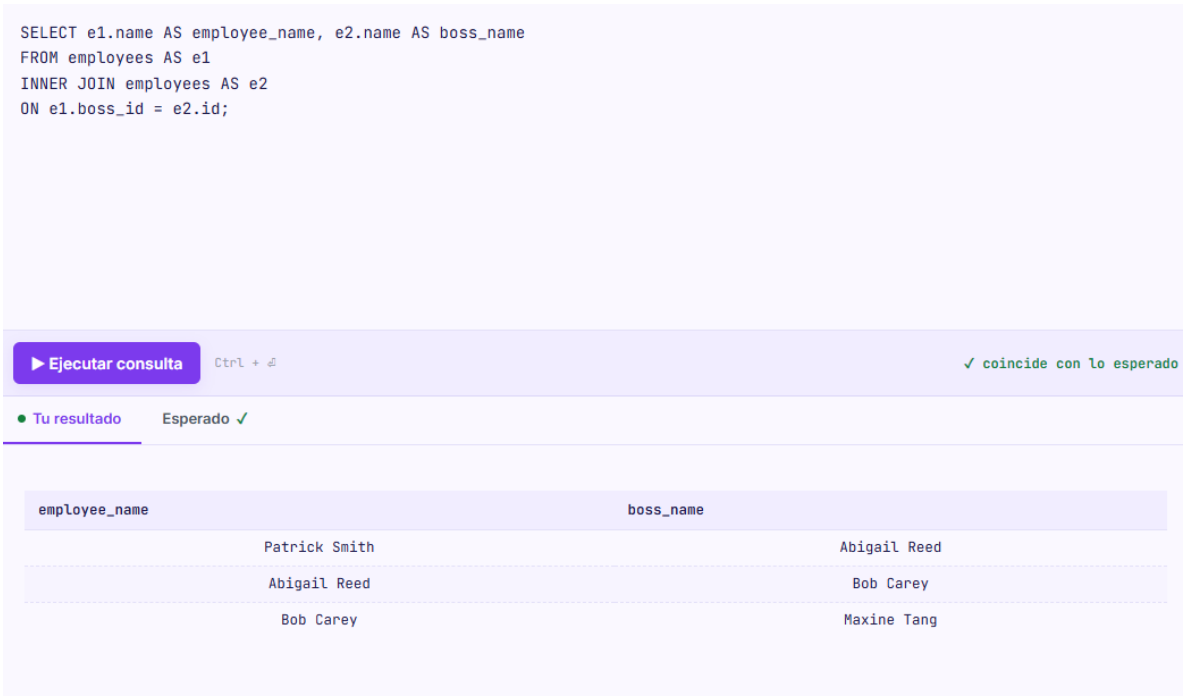
SELECT e1.name AS employee_name, e2.name AS boss_name

FROM employees AS e1

INNER JOIN employees AS e2

ON e1.boss_id = e2.id;

Resultado:



Interpretación: La consulta trata a `employees` como dos instancias virtuales: `e1` (en el rol de empleados subordinados) y `e2` (en el rol de superiores o jefes). Muestra 3 registros con las dependencias directas: Patrick Smith supervisado por Abigail Reed, Abigail Reed supervisada por Bob Carey, y Bob Carey supervisado por Maxine Tang.

Aprendizaje: Se comprendió que una tabla puede relacionarse directamente consigo misma para modelar jerarquías o estructuras en árbol; en este caso, los alias de tabla dejan de ser solo una conveniencia de escritura para volverse un requisito sintáctico imprescindible que le permite al motor distinguir entre la fila de origen y la fila de destino.

Aplicación en el mundo real: En las organizaciones es la base para resolver estructuras organizacionales, líneas de mando y trazabilidad; por ejemplo, mapear organigramas empresariales completos (empleado y supervisor), rastrear programas de referidos donde un cliente invita a otro cliente (`FROM clientes AS c1 INNER JOIN clientes AS c2 ON c1.invitado_por_id = c2.id`), o crear cadenas de sustitución de insumos o componentes en inventario.

Lección 28 LIKE

Objetivo del ejercicio:

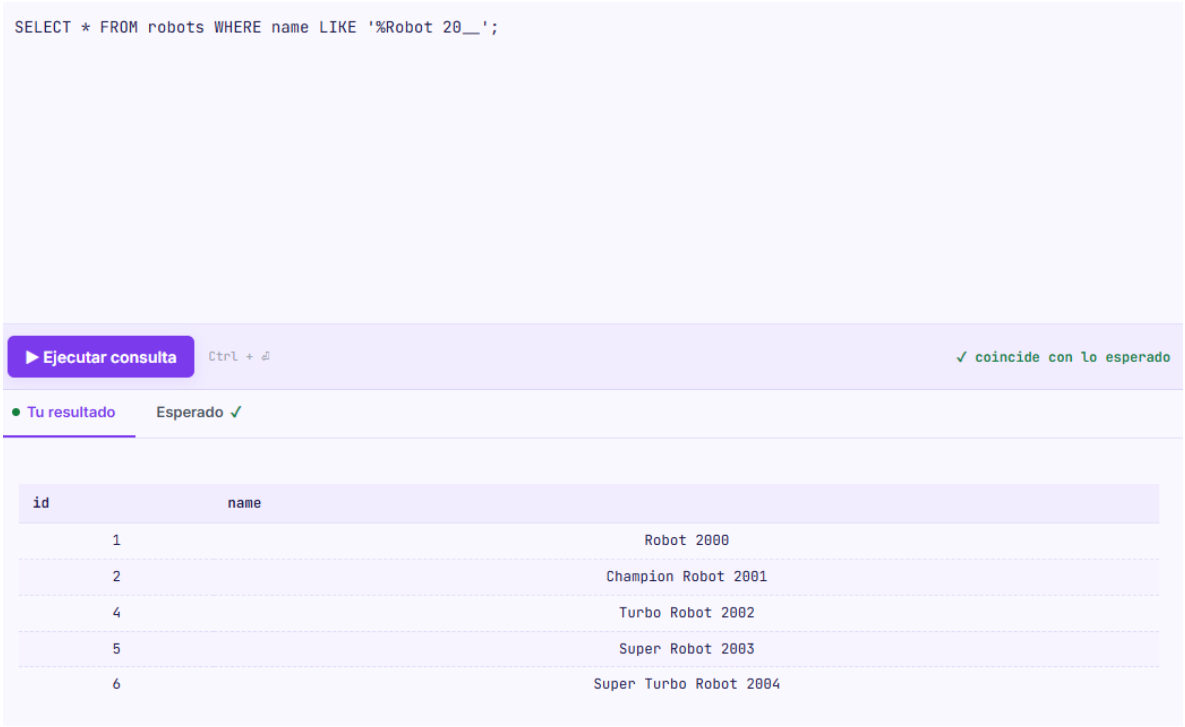
- ✓ Obtener todos los registros de la tabla `robots` cuyo campo `name` contenga el término "Robot" seguido de un año comprendido entre el 2000 y el 2099 (por ejemplo, 2015 es válido, pero 2123 no).

Concepto SQL: Búsqueda de patrones en cadenas de texto mediante el operador `LIKE`, combinando el comodín de longitud variable `%` (cero o más caracteres) con dos comodines posicionales de longitud fija _ (un único carácter por cada guion bajo).

Consulta ejecutada:

```
SELECT * FROM robots WHERE name LIKE '%Robot 20__';
```

Resultado:



Interpretación: El patrón `%Robot 20__` permite cualquier prefijo antes de la palabra "Robot" (`%`), exige un espacio, fija los dos primeros dígitos del año en "20" y reserva exactamente dos posiciones con guiones bajos (__) para representar las décadas y unidades. Esto devuelve 5 registros que cumplen el criterio: Robot 2000, Champion Robot 2001, Turbo Robot 2002, Super Robot 2003 y Super Turbo Robot 2004.

Aprendizaje: Se comprendió la versatilidad de combinar ambos comodines en una sola expresión: mientras `%` ofrece flexibilidad para coincidencias parciales donde la cantidad de caracteres puede variar, _ impone una restricción de longitud exacta carácter por carácter, permitiendo validar formatos estrictos como números de cuatro cifras.

Aplicación en el mundo real: En las organizaciones es fundamental para motores de búsqueda interna y validación de formatos; por ejemplo, buscar clientes por apellido que contenga una raíz (`WHERE apellido LIKE '%Gonzál%'`), validar números de identificación o placas vehiculares con estructuras fijas (`WHERE placa LIKE 'AB___'`), o filtrar números de teléfono o códigos postales según prefijos específicos.

Lección 29 CASE

Objetivo del ejercicio:

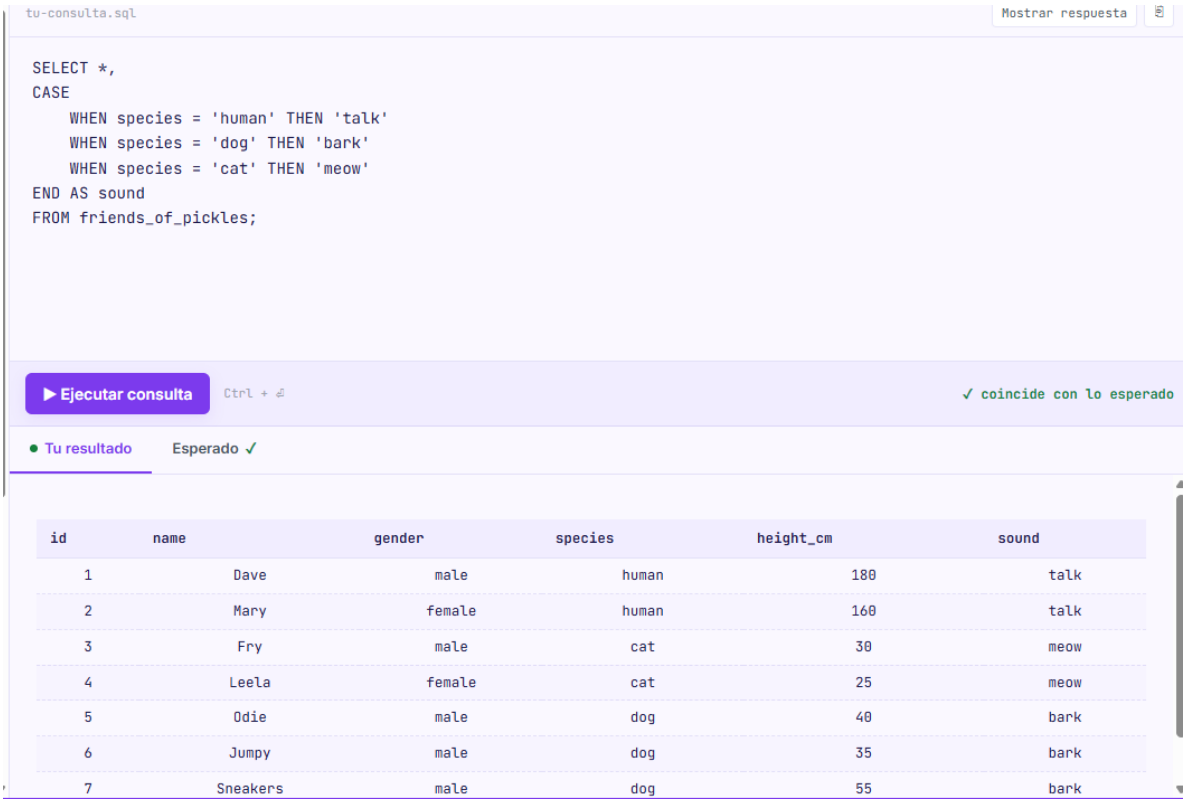
- ✓ Devolver todos los registros de la tabla `friends_of_pickles` agregando una nueva columna calculada llamada `sound` que retorne "talk" para humanos, "bark" para perros y "meow" para gatos en función de la especie.

Concepto SQL: Lógica condicional en la proyección mediante la expresión `CASE WHEN ... THEN ... END`, combinada con la asignación de un alias de columna (`AS sound`).

Consulta ejecutada:

```
SELECT *,
CASE
   WHEN species = 'human' THEN 'talk'
   WHEN species = 'dog' THEN 'bark'
   WHEN species = 'cat' THEN 'meow'
END AS sound
FROM friends_of_pickles;
```

Resultado:



Interpretación: La consulta evalúa dinámicamente el valor del campo `species` fila por fila: asigna `talk` a Dave y Mary (humanos), `meow` a Fry y Leela (gatos), y `bark` a Odie, Jumpy y Sneakers (perros), proyectando todas las columnas originales (`*`) junto a la nueva columna derivada `sound`.

Aprendizaje: Se comprendió cómo implementar estructuras condicionales (similares a `if / else if` en otros lenguajes de programación) directamente dentro del `SELECT`, permitiendo transformar, clasificar o enriquecer los datos devueltos en tiempo de ejecución sin alterar el esquema de la tabla.

Aplicación en el mundo real: En las organizaciones es fundamental para la segmentación y categorización de datos en reportes analíticos; por ejemplo, para clasificar clientes según su volumen de compras (`CASE WHEN compras > 10000 THEN 'VIP' ELSE 'Regular' END`), agrupar edades por rangos generacionales, traducir códigos de estado internos a etiquetas legibles para el negocio (`CASE WHEN estado = 'A' THEN 'Activo' ...`), o estandarizar métricas financieras.

Lección 30 SUBSTR

Objetivo del ejercicio:

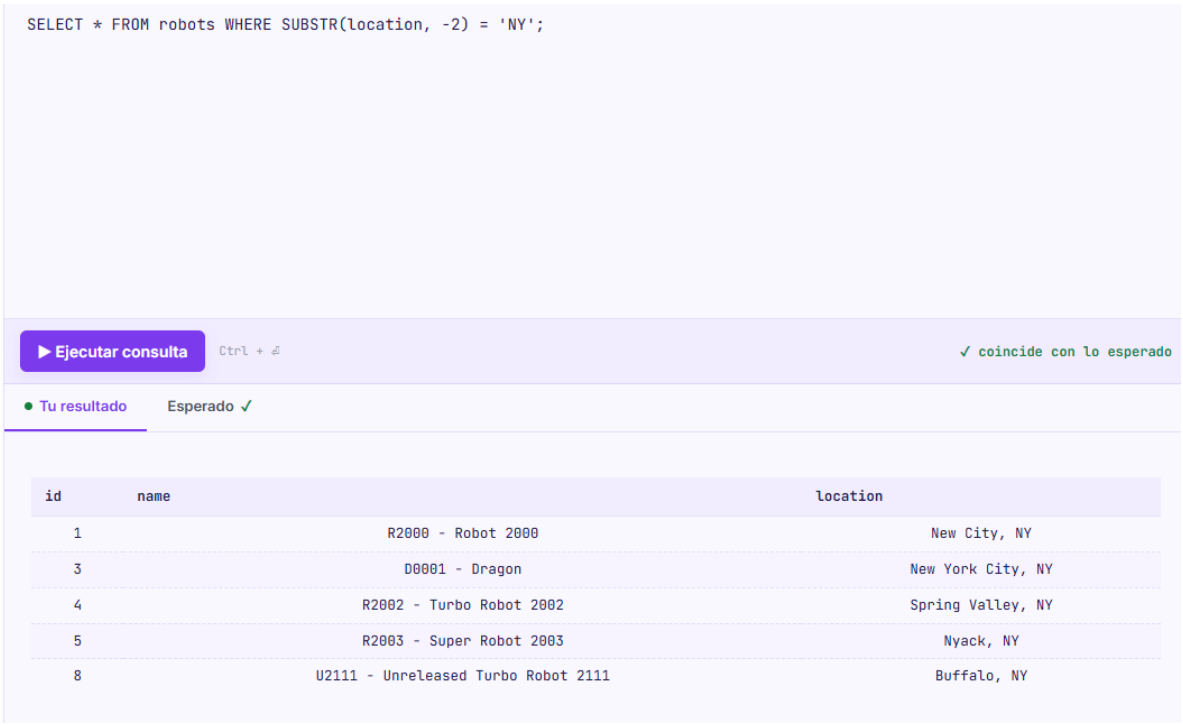
- ✓ Devolver todos los registros de la tabla `robots` cuya ubicación (`location`) corresponda al estado de "NY".

Concepto SQL: Manipulación y extracción de cadenas de texto mediante la función `SUBSTR(columna, indice, [longitud])`, utilizando un índice negativo para posicionarse y

extraer caracteres desde el final de la cadena de derecha a izquierda.
Consulta ejecutada:

```
SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';
```

Resultado:



Interpretación: La función `SUBSTR(location, -2)` analiza los últimos 2 caracteres de cada registro en la columna `location`. Al aplicar la igualdad `= 'NY'`, devuelve 5 robots cuyas ciudades terminan con ese sufijo estatal: R2000 (New City, NY), D0001 (New York City, NY), R2002 (Spring Valley, NY), R2003 (Nyack, NY) y U2111 (Buffalo, NY).

Aprendizaje: Se comprendió cómo recortar y aislar fragmentos específicos de texto sin importar la longitud total del registro, entendiendo que el uso de valores de índice negativos permite una extracción rápida y posicional desde el final de una cadena hacia atrás.

Aplicación en el mundo real: En las organizaciones es fundamental para limpiar y procesar campos no normalizados o semiestructurados; por ejemplo, extraer prefijos o sufijos de códigos de factura o números de serie (`SUBSTR(codigo, 1, 3)`), obtener los últimos 4 dígitos de tarjetas bancarias o identificaciones (`SUBSTR(tarjeta, -4)`), o aislar extensiones de archivos almacenados en la base de datos.

Lección 31 COALESCE

Objetivo del ejercicio:

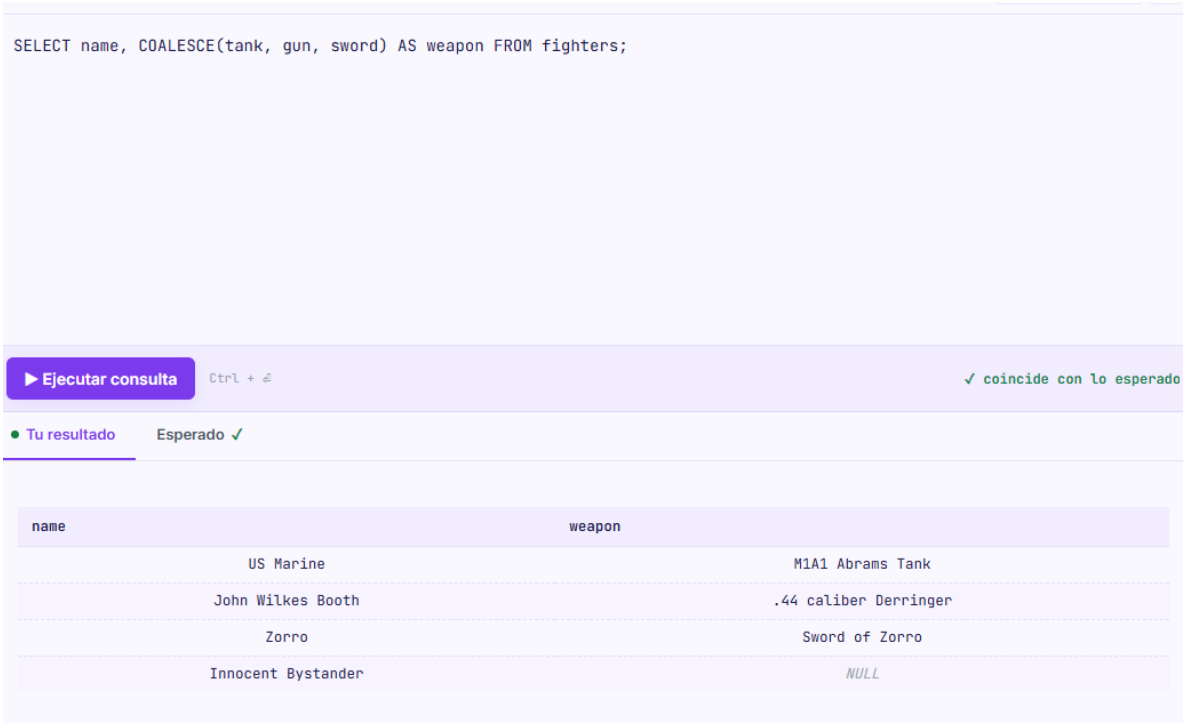
- ✓ Obtener el nombre de cada combatiente (`name`) junto con su arma prioritaria (`weapon`), evaluando las columnas en orden de importancia (`tank`, luego `gun`, y finalmente `sword`) para tomar el primer valor disponible que no sea nulo.

Concepto SQL: Manejo de valores nulos y asignación de prioridades mediante la función `COALESCE(columna1, columna2, ...)`, la cual evalúa sus argumentos secuencialmente y devuelve el primer valor distinto de `NULL`.

Consulta ejecutada:

```
SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;
```

Resultado:



Interpretación:

La consulta revisa cada fila de la tabla `fighters` siguiendo el orden definido en la función:

- Para **US Marine**, al tener valor en `tank`, devuelve `M1A1 Abrams Tank`.
- Para **John Wilkes Booth**, `tank` es nulo, por lo que pasa a `gun` y retorna `.44 caliber Derringer`.
- Para **Zorro**, tanto `tank` como `gun` son nulos, por lo que toma `sword` devolviendo `Sword of Zorro`.
- Para **Innocent Bystander**, las tres columnas son nulas, por lo que el resultado devuelto en `weapon` es `NULL`.

Aprendizaje: Se comprendió la utilidad de `COALESCE` para establecer cadenas de respaldo (*fallbacks*) y jerarquías entre columnas, garantizando la recuperación del primer dato válido sin necesidad de escribir múltiples sentencias `CASE WHEN ... IS NOT NULL` complejas

Aplicación en el mundo real: En las organizaciones es ampliamente utilizada para consolidar información de contacto o datos fiscales donde existen alternativas; por ejemplo, seleccionar el canal de comunicación preferente (`COALESCE(celular, telefono_casa, telefono_oficina, 'Sin contacto')`), determinar el precio efectivo aplicable a una venta (`COALESCE(precio_oferta, precio_regular)`), o rellenar campos vacíos con un texto por defecto para reportes ejecutivos.